



Computational Geometry



计算几何教程



前言

- 这个教程会从矢量开始讲起，学习有关直线、三角形、多边形、圆之知识。
- 教程在下一部分会扩展所学的概念，展开三维计算几何的讲解。
- 最后讲解一些计算几何算法和杂题。

计算几何的恶心之处

- 有精度误差。将在下一页详述。
- 需要讨论各种边界情况。后面所介绍的算法，有些对于边界情况的处理很完美，不需要再做讨论；有些则不然，需要自行处理边界情况。
- 代码长。

精度误差

- 计算几何问题中，很多时候需要繁杂的浮点运算和三角函数运算，这样会产生人神共愤的精度问题。
- 因此，我们采取以下措施尽量避免精度误差（其中 ϵ 是个小量，多取 10^{-8} ）：
 - $a = b \Leftrightarrow |a - b| < \epsilon$
 - $a < b \Leftrightarrow a - b < -\epsilon$
 - $a \leq b \Leftrightarrow a - b < \epsilon$



二维矢量



矢量

- 既有大小又有方向的量。
- 又称为向量。
- 大家初中都毕业了我就不多说了。

矢量的表示

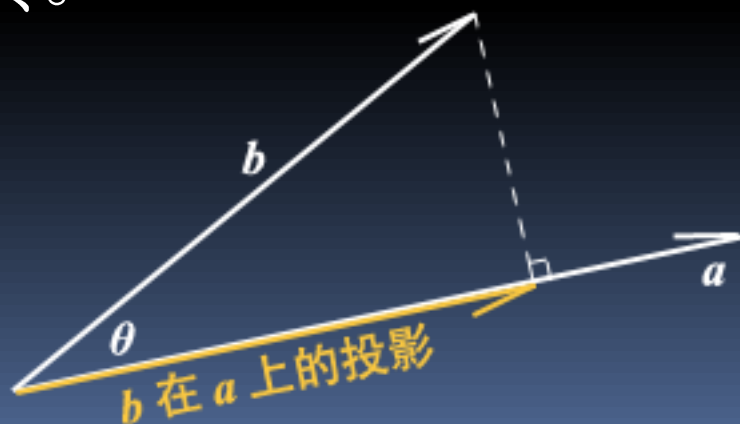
- 在 n 维空间下，矢量经常被表达为 n 个数的元组 $\mathbf{a} = (a_1, a_2, \dots, a_n)$ 。
- 在二维空间下则以 (x, y) 一对整数表示。
- 在高等代数中， n 维矢量一般表示为列矢量的形式 $(a_1 \ a_2 \ \dots \ a_n)^T$ ，即 $n \times 1$ 的矩阵。

点积

- 两个 n 维矢量的点积是一个标量，有
$$(a_1, a_2, \dots, a_n) \cdot (b_1, b_2, \dots, b_n) \stackrel{\text{def}}{=} a_1 b_1 + a_2 b_2 + \dots + a_n b_n。$$
- 由此，矢量的模（即长度）定义为
$$|\mathbf{a}| \stackrel{\text{def}}{=} \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\mathbf{a}^2}。$$
- 点积满足交换律。

夹角

- 点积之另一定义为 $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$ ，其中 θ 为 $\mathbf{a}$ 和 $\mathbf{b}$ 之夹角。
- 如下图， $\mathbf{a}$ 与 $\mathbf{b}$ 的点积的值实际上就是 $\mathbf{a}$ 的模乘 $\mathbf{b}$ 在 $\mathbf{a}$ 上的投影的“模”，但是若其投影与 $\mathbf{a}$ 方向相反则为负。



矢量的垂直

- 垂直的矢量有 $\theta = \pi/2$, 即 $\cos\theta = 0$.
- 所以 $\mathbf{a}$ 和 $\mathbf{b}$ 垂直定义为 $\mathbf{a} \cdot \mathbf{b} = 0$ 。

矢量的缩放

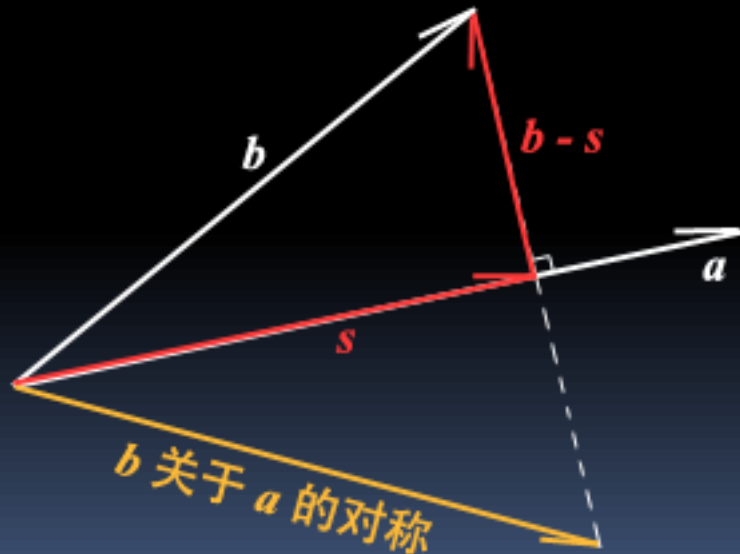
- 矢量 $\frac{\mathbf{a}}{|\mathbf{a}|}$ 是与 $\mathbf{a}$ 同向的**单位矢量**，即模长是 1 的矢量。
- 所以与 $\mathbf{a}$ 同向，但长度是 l 的矢量，为 $l \frac{\mathbf{a}}{|\mathbf{a}|}$ 。
- 而与 $\mathbf{a}$ 共线但方向相反，长度是 l 的矢量为 $-l \frac{\mathbf{a}}{|\mathbf{a}|}$ 。

矢量的投影

- $\mathbf{b}$ 在 $\mathbf{a}$ 上的投影的“模”是 $\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}|}$ 。
- 所以 $\mathbf{b}$ 在 $\mathbf{a}$ 上的投影即为 $\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}|} \frac{\mathbf{a}}{|\mathbf{a}|} = \mathbf{a} \frac{\mathbf{a} \cdot \mathbf{b}}{a^2}$ 。

矢量的对称

- 记 $\mathbf{b}$ 在 $\mathbf{a}$ 上的投影为 $\mathbf{s} = \mathbf{a} \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a}^2}$ 。
- 则 $\mathbf{b}$ 关于 $\mathbf{a}$ 的对称为 $\mathbf{b} - 2(\mathbf{b} - \mathbf{s}) = 2\mathbf{s} - \mathbf{b} = 2\mathbf{a} \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a}^2} - \mathbf{b}$ 。

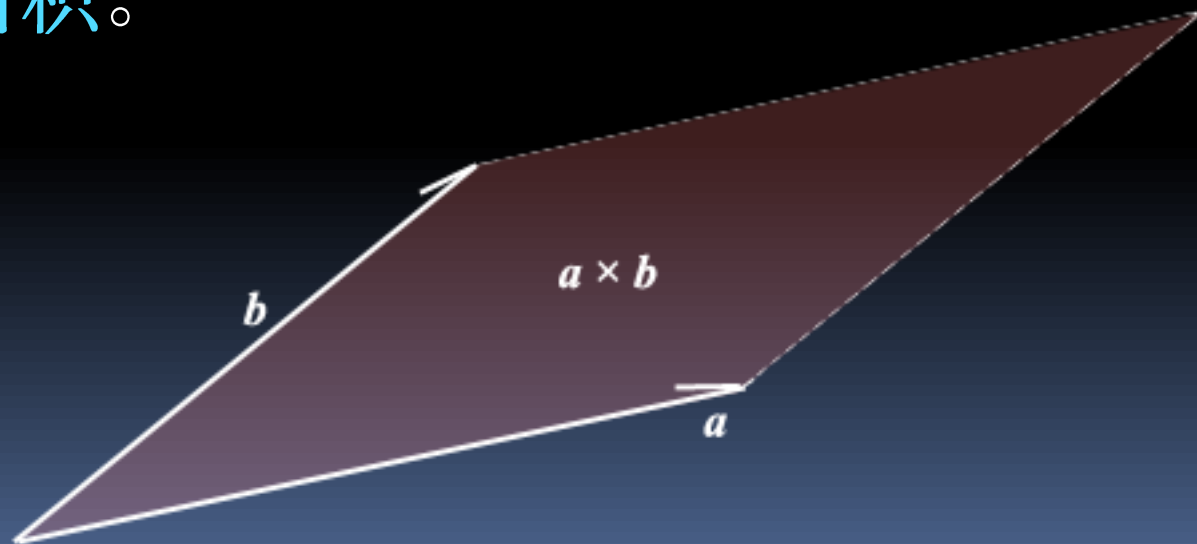


二维叉积

- 两个二维矢量的二维叉积是一个标量，定义为 $(x_1, y_1) \times (x_2, y_2) \stackrel{\text{def}}{=} x_1 y_2 - x_2 y_1$ 。
- 二维叉积满足逆交换律： $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$ 。
- 二维叉积是计算几何中最常用的概念。

有向面积

- 经过计算可以知道， $\mathbf{a}$ 和 $\mathbf{b}$ 所成的平行四边形的面积即为 $|\mathbf{a} \times \mathbf{b}|$ 的值。
- 去掉绝对值符号，二维叉积则定义为有向面积。



有向面积的符号

- 伸出右手，将四指由 $\mathbf{a}$ 沿小于平角转到 $\mathbf{b}$ 。若拇指指向纸面上方，则 $\mathbf{a} \times \mathbf{b}$ 为正，否则为负。
- 若 $\mathbf{a}$ 与 $\mathbf{b}$ 共线，则 $\mathbf{a} \times \mathbf{b} = 0$ 。
- 利用有向面积可以非常简单自然地计算简单多边形的面积。后面我们会详述这一点。

点积和二维叉积的方向性

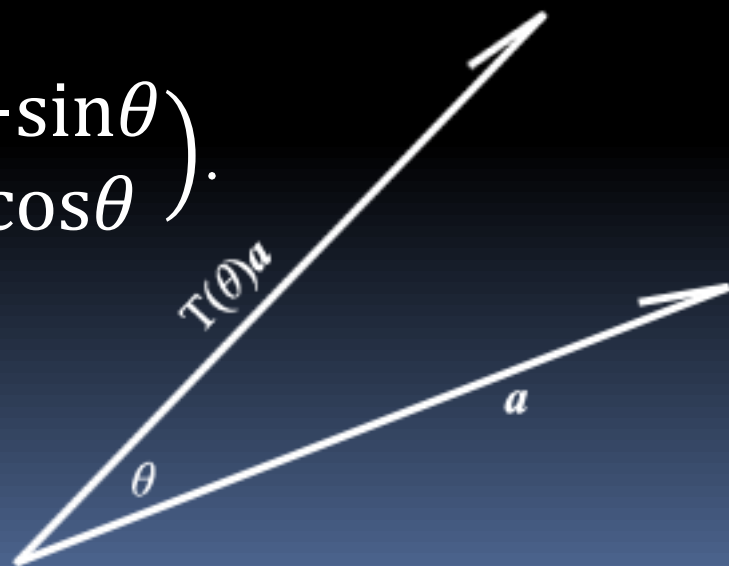
- 利用点积可以判断矢量的前后。
- 利用二维叉积可以判断矢量的左右。

二维矢量的旋转

- 将矢量看做列矢量，即 2×1 的矩阵。
- 则将矢量 $\mathbf{a}$ 逆时针旋转 θ 之后的矢量为

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \mathbf{a}.$$

- 记矩阵 $T(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$.



二维矢量的极角

- 极角指示矢量的方向，以 x 轴正半轴逆时针转过的角度来指示。
- 即矢量 (x, y) 的极角为 $\text{atan2}(y, x)$ 。
- 极角的取值范围是 $]-\pi, \pi]$ 。



二维计算几何



约定

- 虽然点与矢量有着本质的不同，但是为了方便起见，本教程中坐标为 (x, y) 的点也作为矢量 (x, y) 使用，反之亦然。
- 比如， A 和 B 的中点 $M = \frac{A+B}{2}$.

直线

- 在二维空间中，直线用两个相异点 A 和 B 表示。则 $\forall \lambda \in \mathbb{R}, \lambda A + (1 - \lambda)B$ 代表直线上一点。
- 若 $\lambda < 0$ ，则此点在 B 外侧；若 $\lambda > 1$ ，则此点在 A 外侧。
- 视情况，有时也用一个点 P 和一个方向向量 $\mathbf{v}$ 来表示。则 $\forall \lambda \in \mathbb{R}, P + \lambda \mathbf{v}$ 代表直线上一点。

点到直线的距离

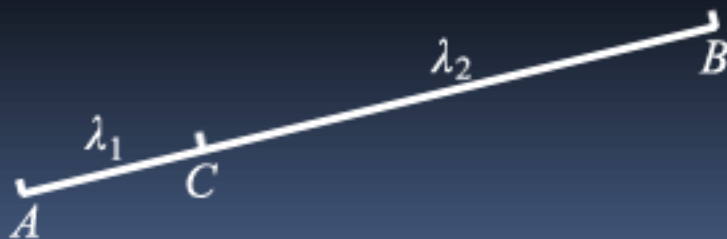
- 若要计算点 P 到直线 AB 的距离，只需要计算 $\overrightarrow{AP}$ 与它到 $\overrightarrow{AB}$ 的投影之差的模即可。
- 即 $\left| \overrightarrow{AP} - \overrightarrow{AB} \frac{\overrightarrow{AB} \cdot \overrightarrow{AP}}{\overrightarrow{AB}^2} \right|$ 。
- $\overrightarrow{AP} - \overrightarrow{AB} \frac{\overrightarrow{AB} \cdot \overrightarrow{AP}}{\overrightarrow{AB}^2}$ 后面称为 P 到 AB 的垂直矢量。注意，这名字是我自己起的。

分点

- 如下图, 若 A, C, B 共线, 且 $\frac{|\overline{AC}|}{|\overline{CB}|} = \frac{\lambda_1}{\lambda_2}$, 则

$$C = \frac{\lambda_2 A + \lambda_1 B}{\lambda_1 + \lambda_2}.$$

- 特别地, 若 C 在 A 外侧, 则 $\lambda_1 < 0$; 若 C 在 B 外侧, 则 $\lambda_2 < 0$ 。此时上式仍然成立。

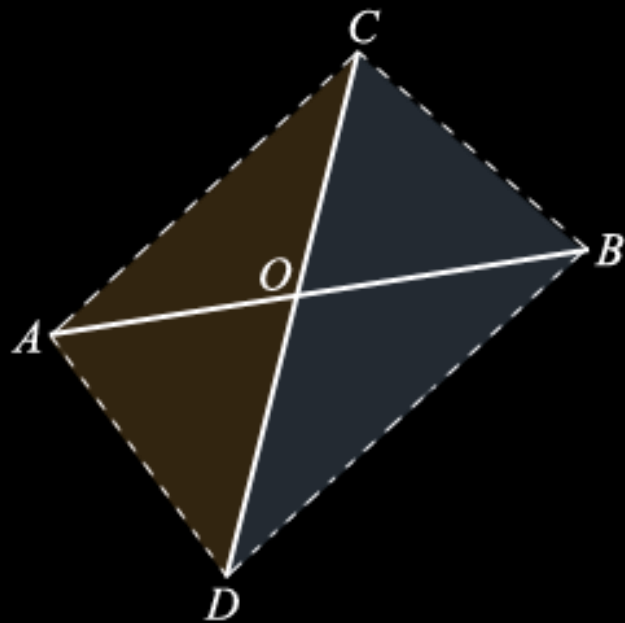


三角形的面积

- 利用二维叉积即得 $S(\Delta ABC) = \frac{|\overrightarrow{AB} \times \overrightarrow{AC}|}{2}$ 。

两直线交点

- 先排除平行或重合的情况。
- $\frac{|\overline{AO}|}{|\overline{OB}|} = \frac{S(\Delta ADC)}{S(\Delta BCD)} = \frac{\overline{AD} \times \overline{AC}}{\overline{BC} \times \overline{BD}}.$
- 计算分点即可得到点 O .
- 注意上式中 A 和 B 叉积顺序相反。
- 按照有向面积，此方法在交点不在线段 AB 上，甚至也不在线段 CD 上时亦成立。

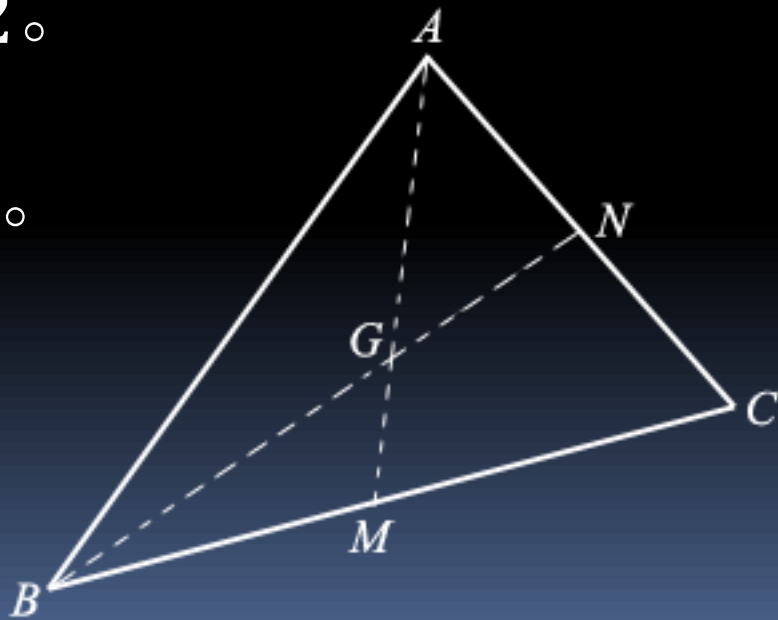


两线段交点

- 求出直线的交点，然后其判断是否在两条线段上即可。
- 若判断两线段是否相交，则要注意平行不一定不相交，两条线段可能有重合部分或重合点。

三角形的重心

- 众所周知，三角形的重心 G 是三条中线的交点，即 AM 与 BN 的交点。
- 而 $M = \frac{B+C}{2}$, $\frac{|AG|}{|GM|} = 2$ 。
- 故 $G = \frac{2M+A}{3} = \frac{A+B+C}{3}$ 。

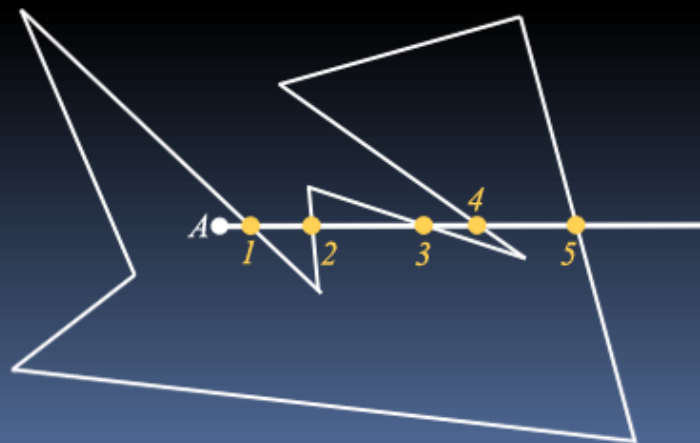


多边形

- 多边形是平面上由有限线段（大于2）组成，且首尾连接起来划出的封闭图形。
- 教程后面的多边形一般指简单多边形，即边不相交的多边形。
- 多边形的记录一般以顺次记录其顶点的方式完成，即顺次记录组成多边形的线段。方向一般为逆时针。

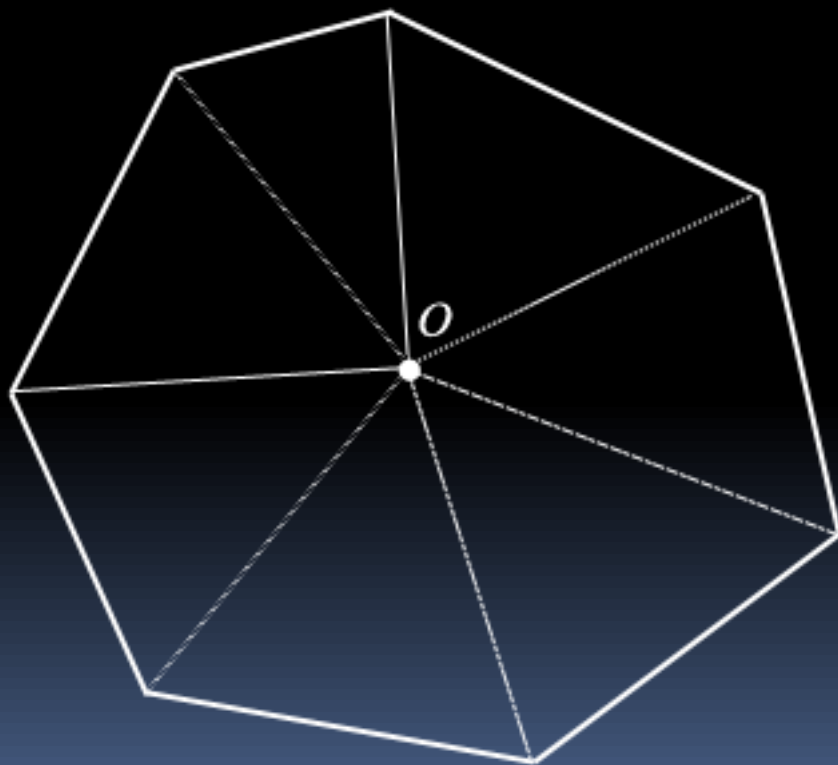
判断点在多边形内外

- 以要判断的点为起点任作一射线，计算该射线与多边形的交点数目。
- 若有偶数个交点则在形外，否则在形内。
- 若与线段在端点处相交或重合，则要进行复杂的判断。此时可另取一射线。



求多边形的面积

- 基本的思路是进行三角剖分。





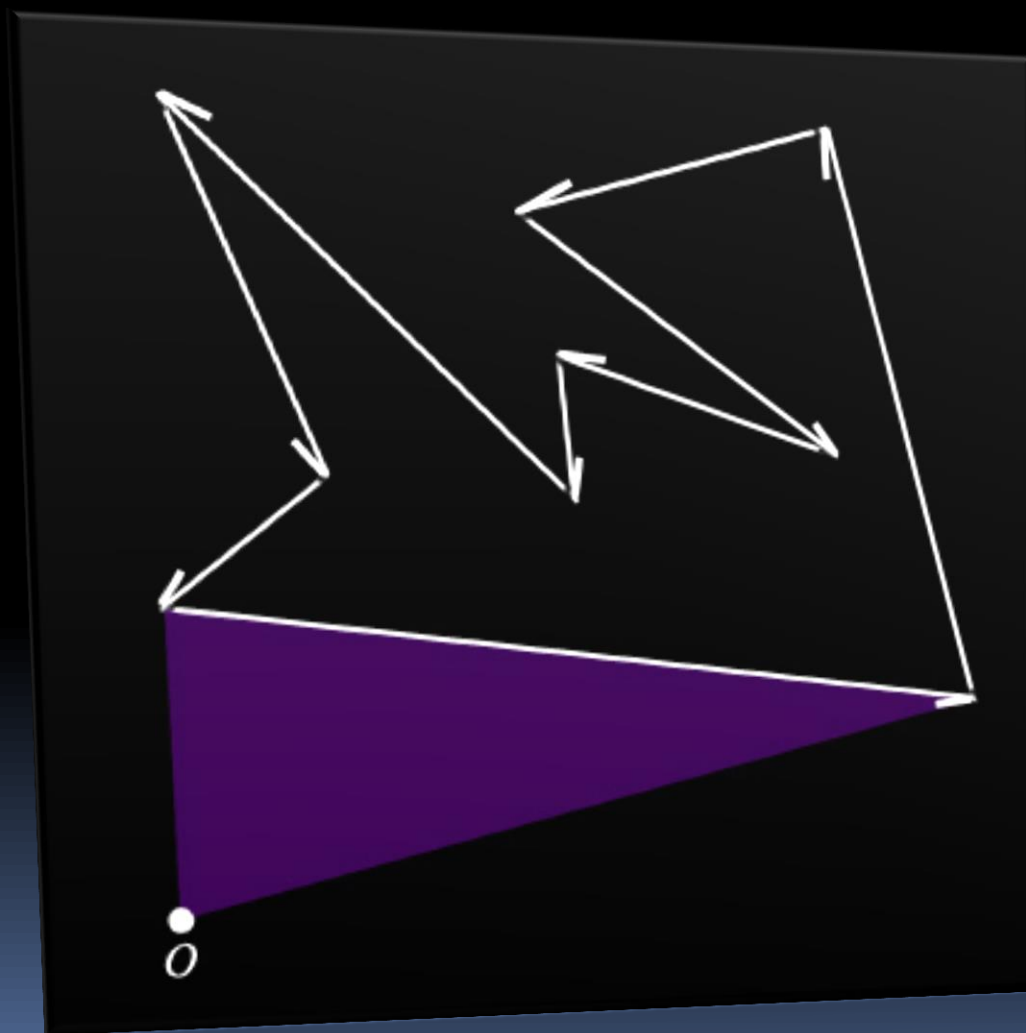
求多边形的面积

- 如果用普通面积的累加来计算的话，三角剖分就变得十分复杂。
 - 但是使用有向面积的话，这个过程就变得简单自然通用。
- 

算法

- 按逆时针方向顺次为各边指定方向。
- 对于每条边 $\overrightarrow{AB}$ ，累加 $\frac{A \times B}{2}$ 的值。
- 最后得到的结果即为多边形的面积。
- 当然也可以累加 $A \times B$ 的值，最后再除以 2。

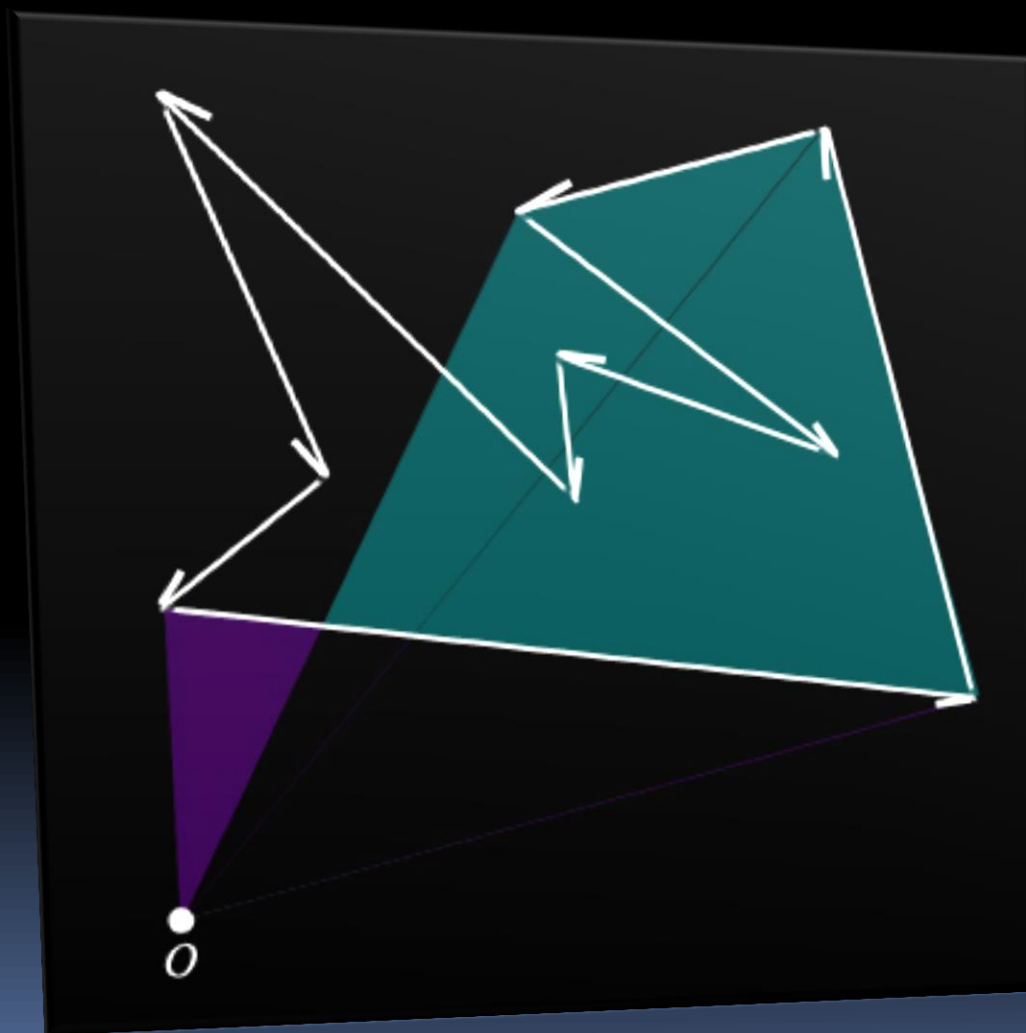
算法演示



算法演示



算法演示



算法演示



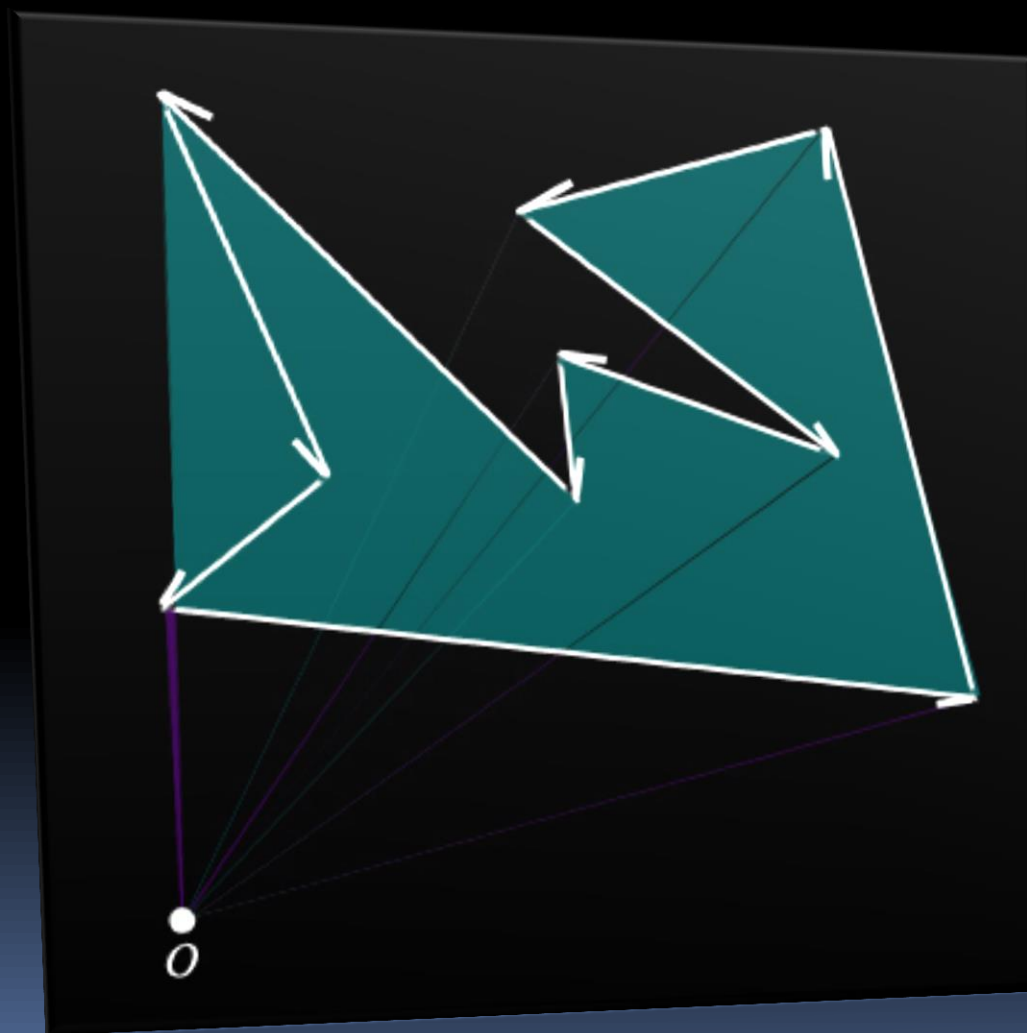
算法演示



算法演示



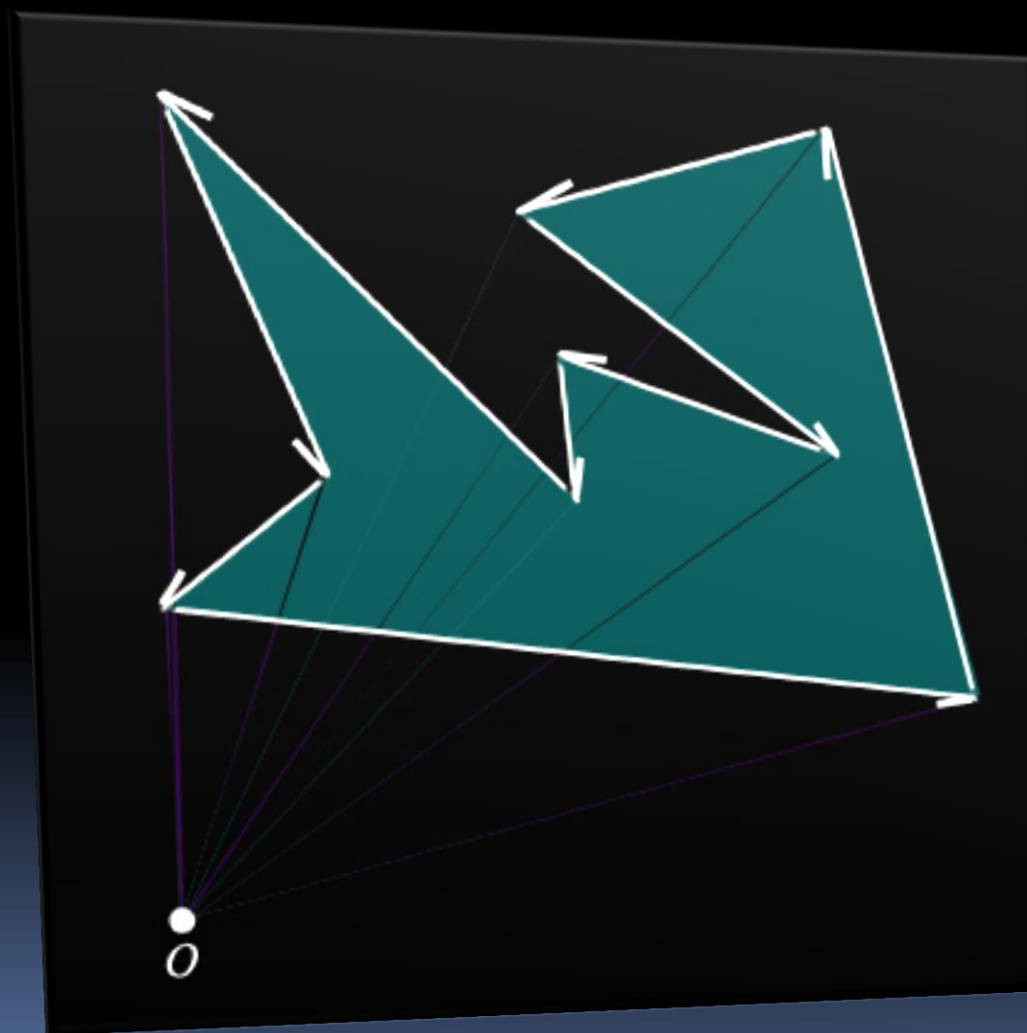
算法演示



算法演示



算法演示



三角剖分

- 三角剖分即将多边形分为若干三角形部分，其中既有“正部分”亦有“负部分”。
- 利用这种方法可以解决很多问题。

例题：求多边形的重心

- 名前通り，给定一个简单多边形，求其重心之所在。

预备知识：求质点组的重心

- n 个位置分别为 $A_1, A_2, \dots, A_n$ ，质量为 $m_1, m_2, \dots, m_n$ 的质点构成的质点组的重心为

$$\frac{\sum_{k=1}^n m_k A_k}{\sum_{k=1}^n m_k}$$

解答

- 利用三角剖分，计算各个三角形的面积和重心，将此三角形化为一个质量为其面积，坐标为其重心的质点。
- 若此三角形属于“负部分”，则其拥有“负质量”。
- 最后计算质点组的重心即可。

凸集

- 对于一个点集 S ，若 $\forall a, b \in S, \lambda \in [0, 1]$ ，有 $\lambda a + (1 - \lambda)b \in S$ ，则称 S 是凸集。
- 说人话的话，就是对于点集中任意两点，以两点为端点的线段上的所有点也属于这个点集。
- 若多边形及其内点是凸集，则称这个多边形为凸多边形。
- 可以证明凸集之交也是凸集。

凸集举例

- 空集
- 直线、射线、线段
- 凸多边形
- 圆及内部、二次曲线及内部
- 若干种原料，每种原料含 A 成分和 B 成分的比例不同；混合这些原料能得到的产品的 A, B 两种成分的比例

凸包

- 对于点集 S ，包含 S 中所有点的最小的凸集称为 S 的凸包。
- 若 S 是有限集，则其凸包是凸多边形。
- 直观来看，把 S 中所有点钉在一个木板上，用一个橡皮筋框起所有点来，一撒手，橡皮筋就表示出了 S 的凸包。

凸包



水平序 Graham 扫描算法

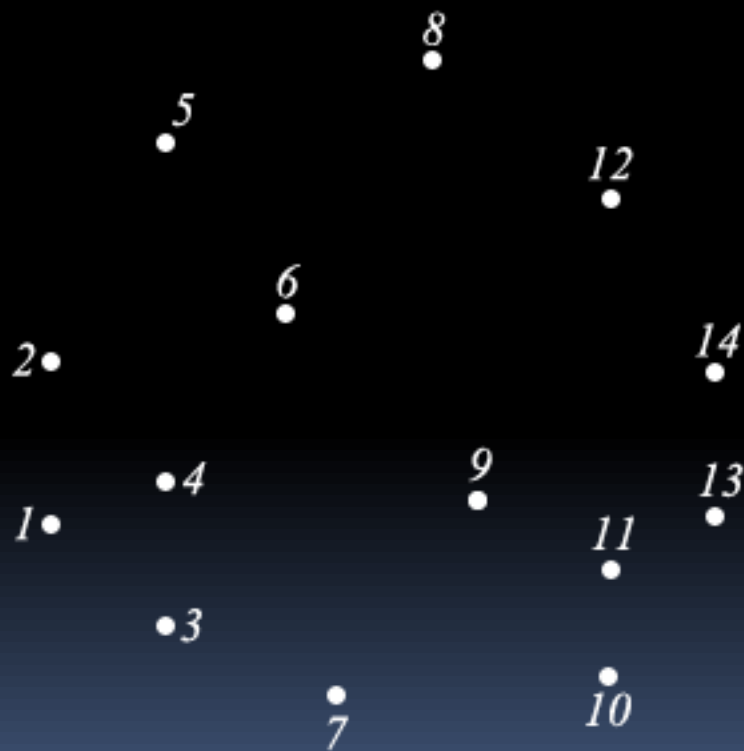
- Graham 扫描算法有水平序和极角序两种。
- 极角序算法能一次确定整个凸包，但是计算极角需要用到三角函数，速度较慢，精度较差，特殊情况较多。
- 水平序算法需要扫描两次，但排序简单，讨论简单，不易出错。

算法流程

- 对顶点按 x 为第一关键字, y 为第二关键字进行排序。
- 准备一个空栈, 并将前两个点压入栈。
- 对于每一个顶点 A , 只要栈顶中还有至少两个顶点, 记栈顶为 T , 栈中第二个为 U 。若 $\overrightarrow{UT} \times \overrightarrow{TA} \leq 0$, 则将 T 弹出。重复此过程。
- 直到上一步不再弹出顶点, 将 A 压入栈。扫描完一遍之后得到凸包的**下凸壳**。
- 将点集倒过来再进行一次, 得到凸包的**上凸壳**, 组合起来即可。

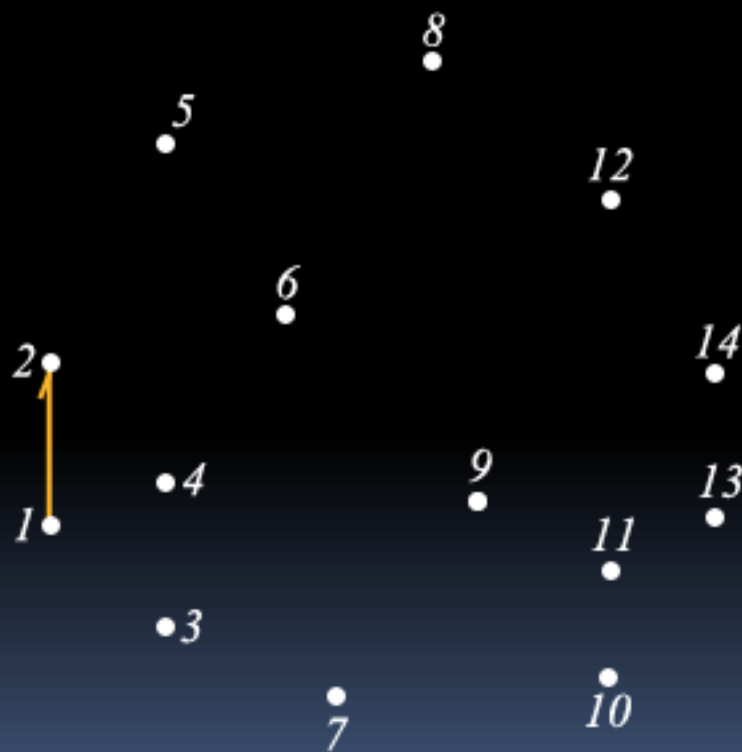
算法演示

首先将所有点排序。



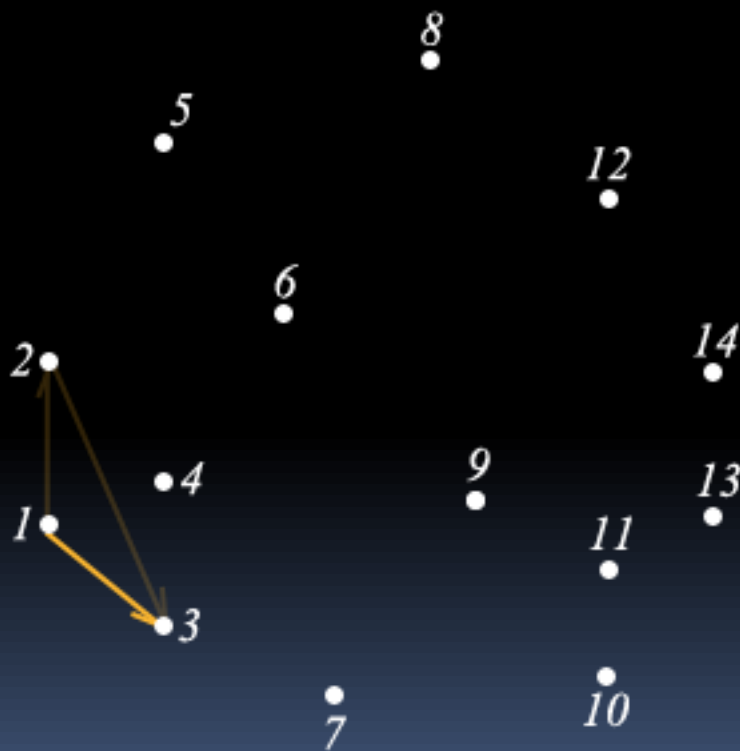
算法演示

将 1 和 2 压入栈。



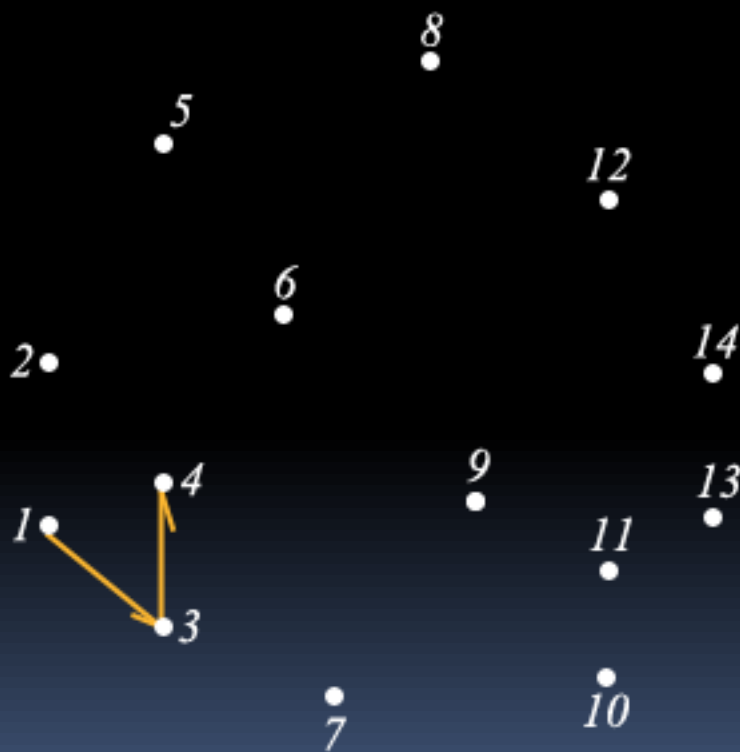
算法演示

2 被弹出栈，3 进栈。



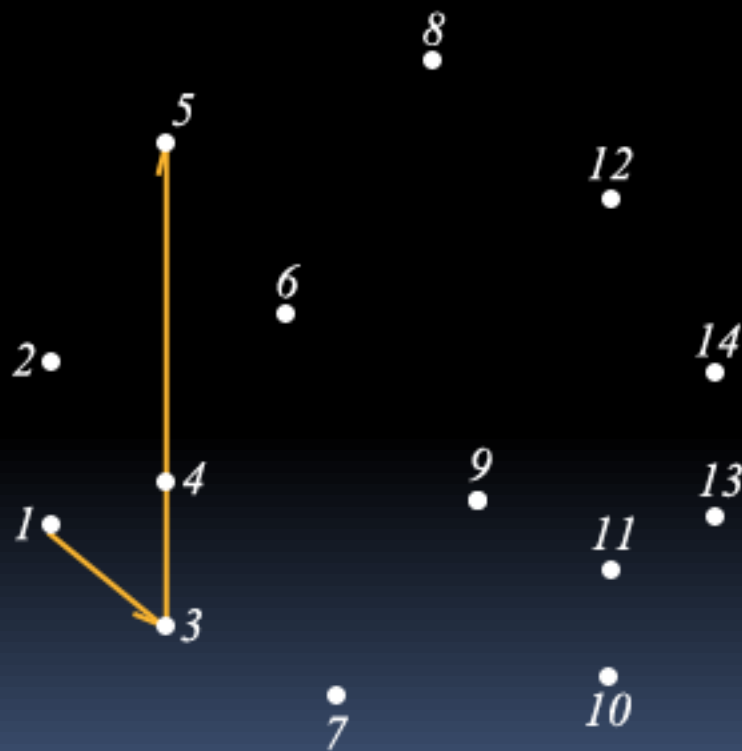
算法演示

没有点被弹出栈，4 进栈。



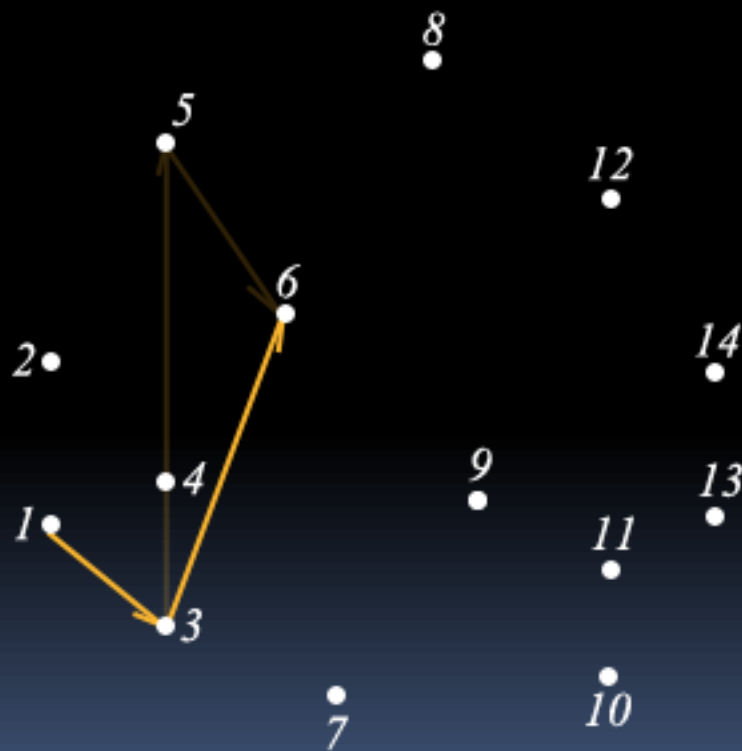
算法演示

4 被弹出栈，5 进栈。



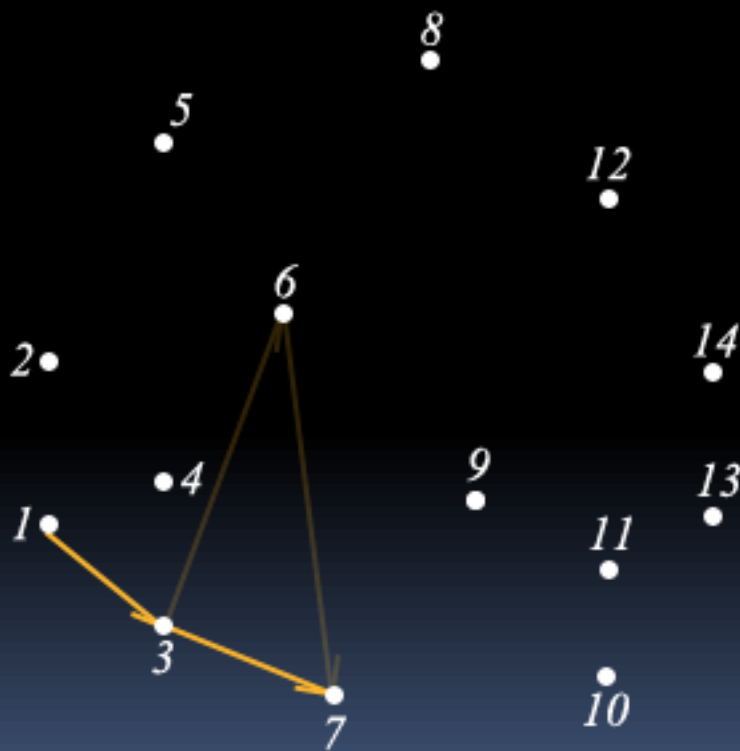
算法演示

5 被弹出栈，6 进栈。



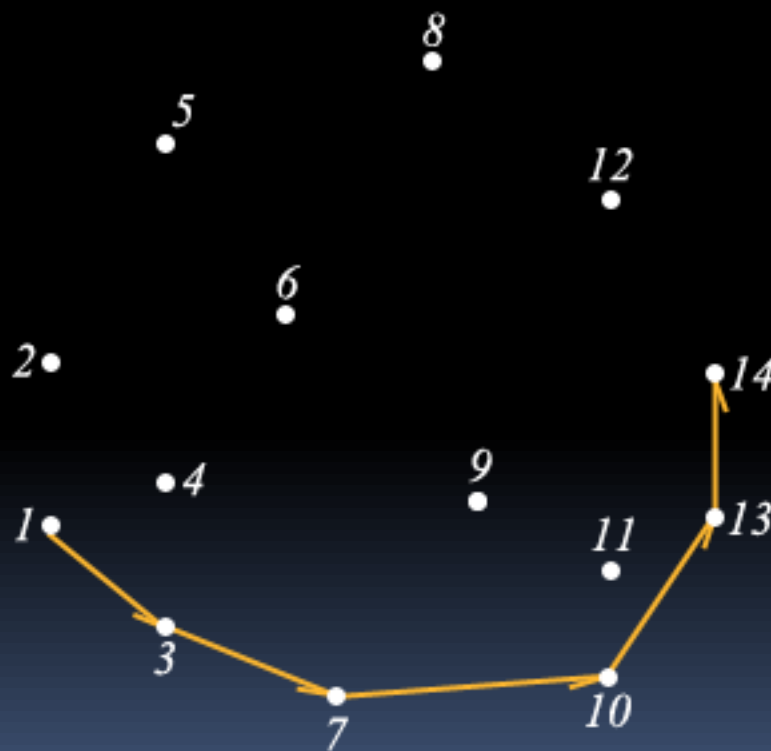
算法演示

6 被弹出栈，7 进栈。



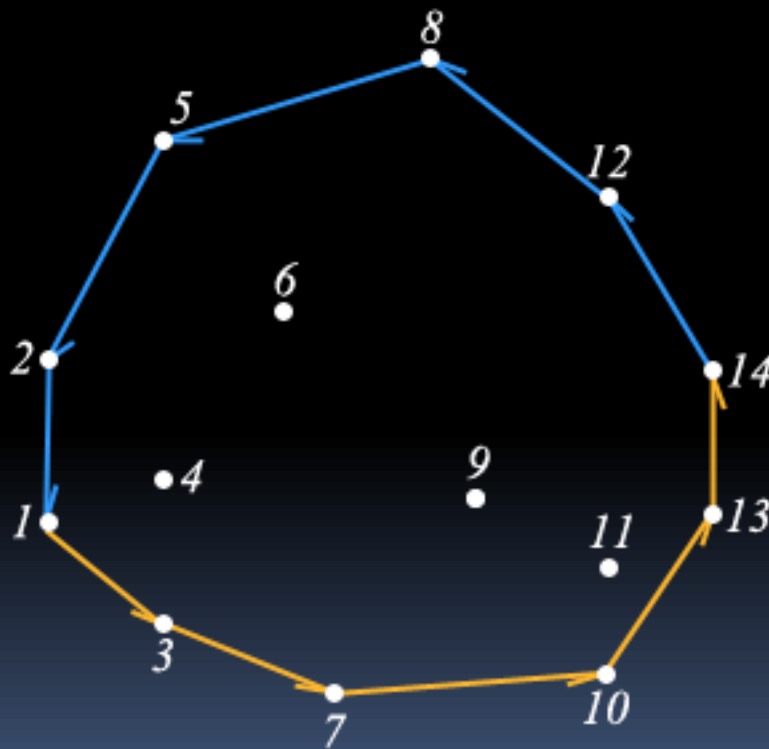
算法演示

如此直到下凸壳被找到。



算法演示

倒序进行扫描找到上凸壳。



Graham 扫描的时间复杂度

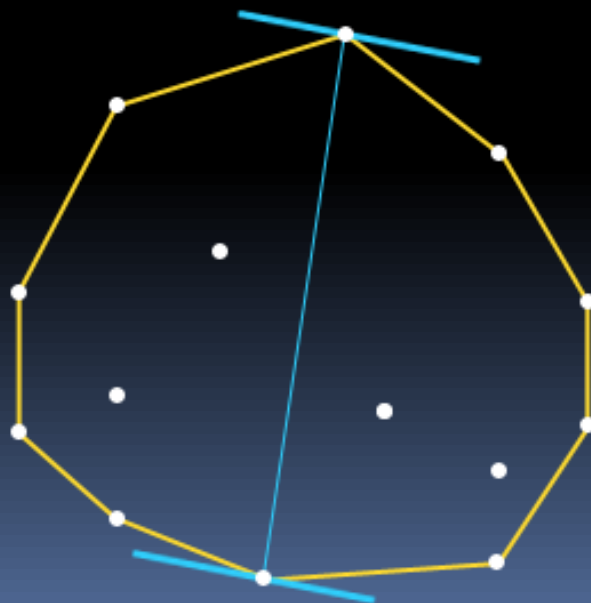
- 这太傻缺了。算法的瓶颈在排序，所以时间复杂度是 $O(N \log N)$ 。
- 若坐标均为整数，可以用基数排序将复杂度优化到 $O(N)$ 。

旋转卡壳算法

- 如何计算 N 个点中，距离最大的两个点的距离？
- 要求 $O(N \log N)$ 的算法。
- 容易证明距离最大的两个点一定是凸包上的两个点，很明显是先求凸包然后做文章啦。

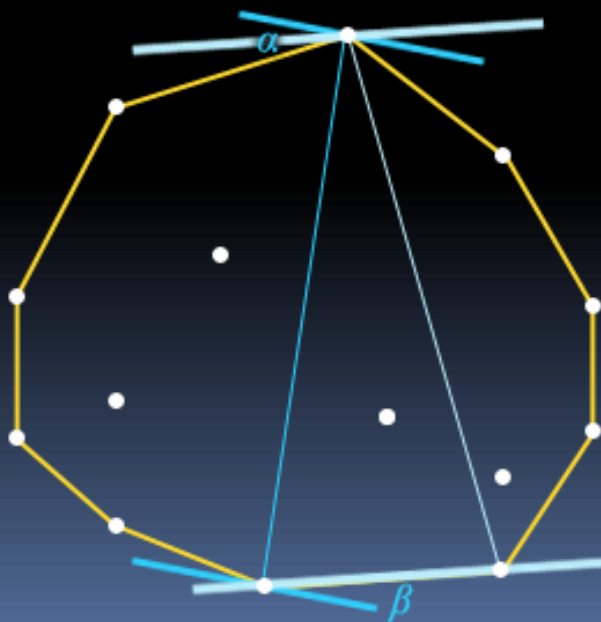
对踵点

- 若凸包上的两个顶点能落在一对平行切线上，则称它们为对踵点。
- 容易证明，答案一定是一一对对踵点之间的距离。



对踵点的枚举

- 从一对对踵点逆时针枚举下一对对踵点的话，只需要观察下图中 α 角和 β 角的大小，选择其中较小的一个转过去就行了。



算法

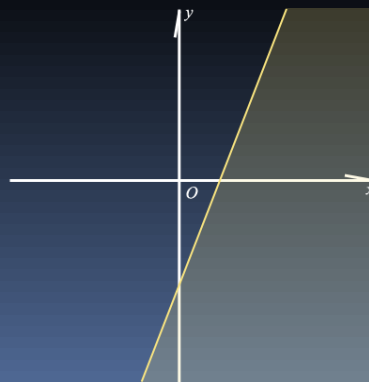
- 最左侧点和最右侧点一定是对踵点。从此点对开始枚举即可。
- 注意不只有“点 - 点”的情况，还有“点 - 边”和“边 - 边”的情况。这些情况下要枚举所有的点对。
- 虽然看上去很简单，但是写起来还是要颇费一番周折的。

旋转卡壳练习题

- POJ 2178 求点集中最远点
- POJ 3608 求两凸包间的最近点
- POJ 2079 求以点集中的点为顶点组成的三角形的最大面积，要求 $O(N^2)$ 算法。

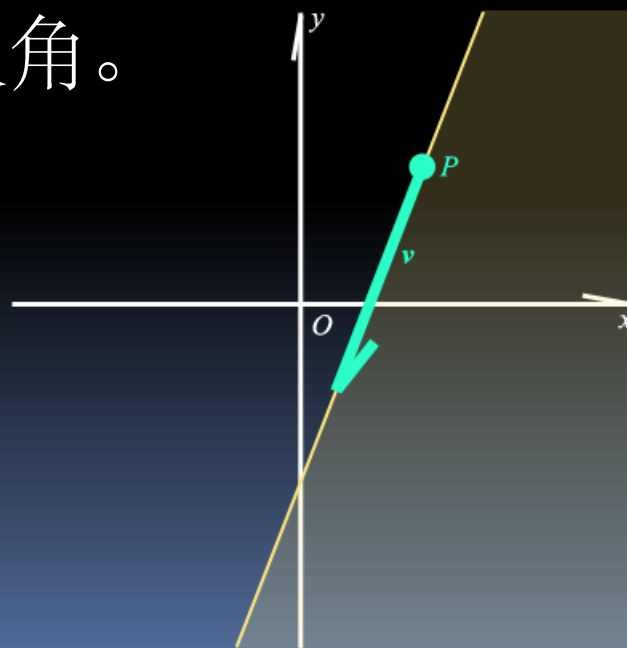
半平面

- 半平面指一条直线的一侧的点构成的点集，一般包含直线上的点。
- 在解析几何中直线多以 $Ax + By + C = 0$ 表示 (其中 A, B, C 是常数, A, B 不全等于 0)。半平面交即以 $Ax + By + C \geq 0$ 表示。



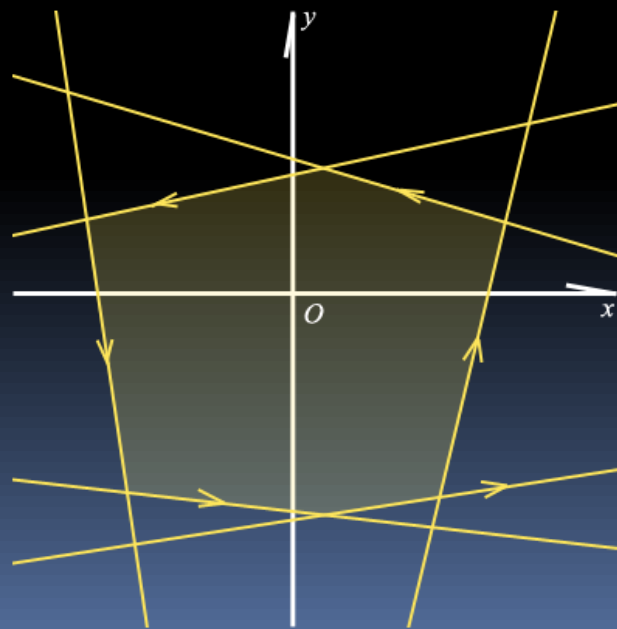
半平面的表示

- 半平面一般用一个点 P 和方向矢量 $\mathbf{v}$ 来代表。满足 $\mathbf{v} \times (\mathbf{A} - \mathbf{P}) \geq 0$ 的点 A 在半平面上。
- $\mathbf{v}$ 的极角即为半平面的极角。



半平面的交

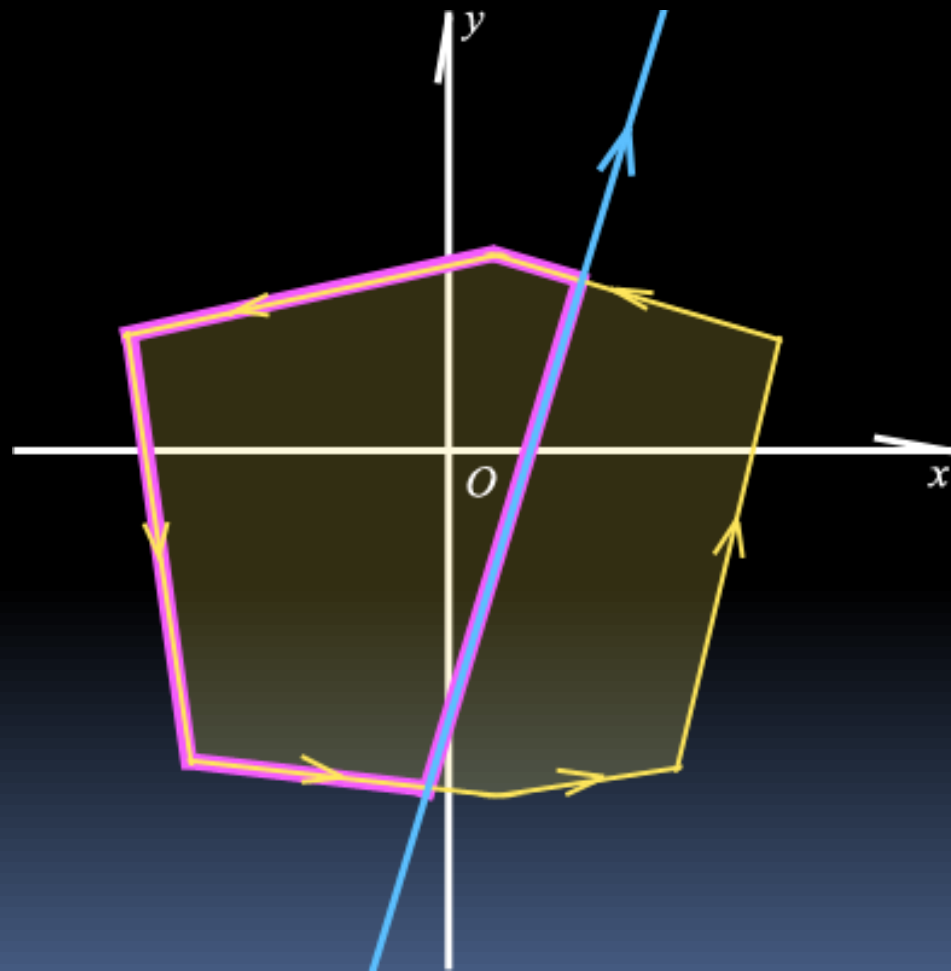
- 半平面是凸集，所以半平面的交也是凸集。
- 有限个半平面的交可能是空集、点、直线、射线、线段、凸多边形、有“开口”的形状甚至一个半平面。



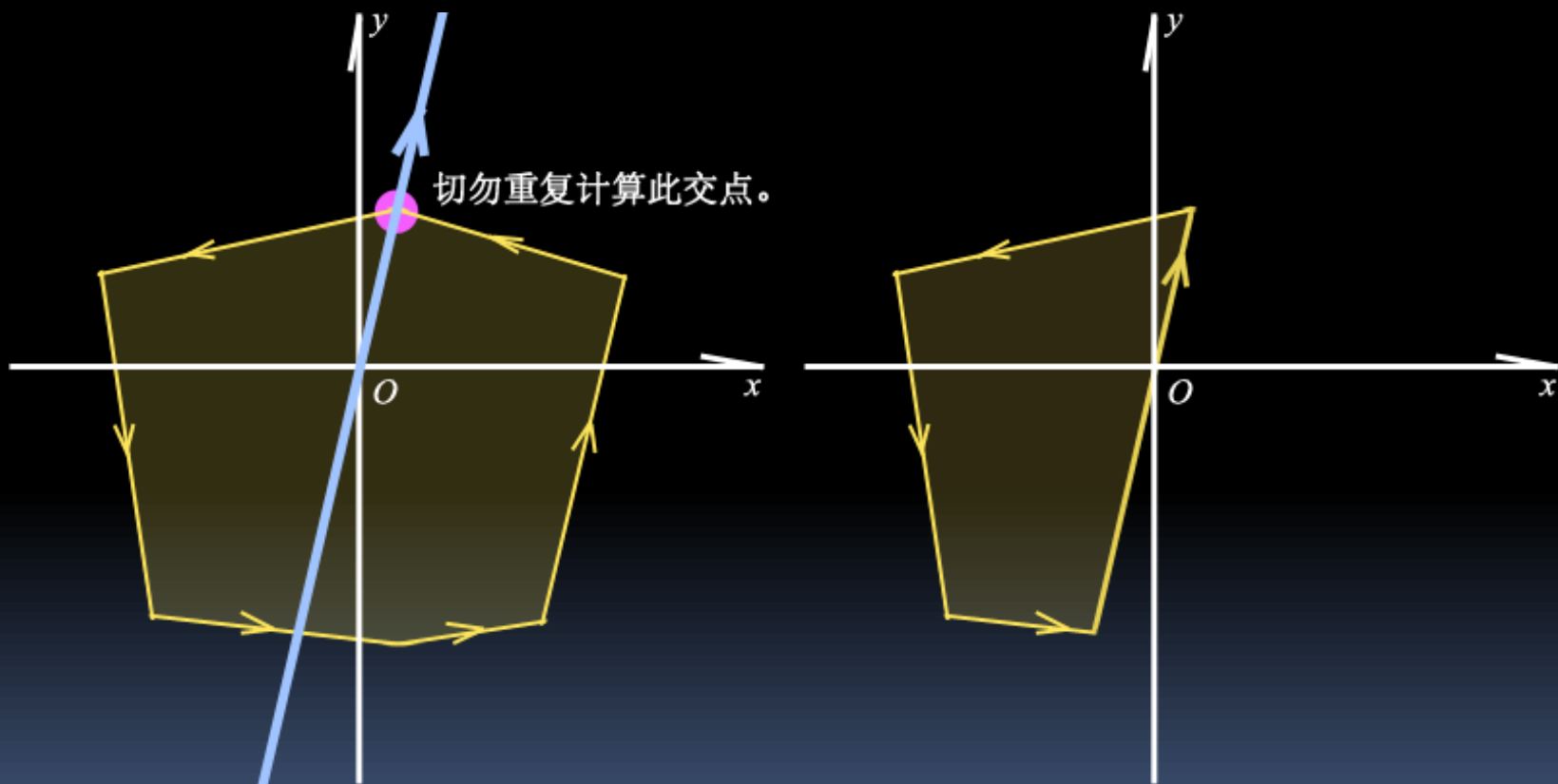
朴素的每次 $O(N)$ 算法

- 为了保证结果是凸多边形或其退化，首先建立一个非~常~大的边框。你懂的。
- 每次顺次枚举多边形的边，与半平面求交，并将交的部分加入新多边形。
- 在**得到**半平面边界与凸多边形所交形成的边时，立即将其加入新多边形。

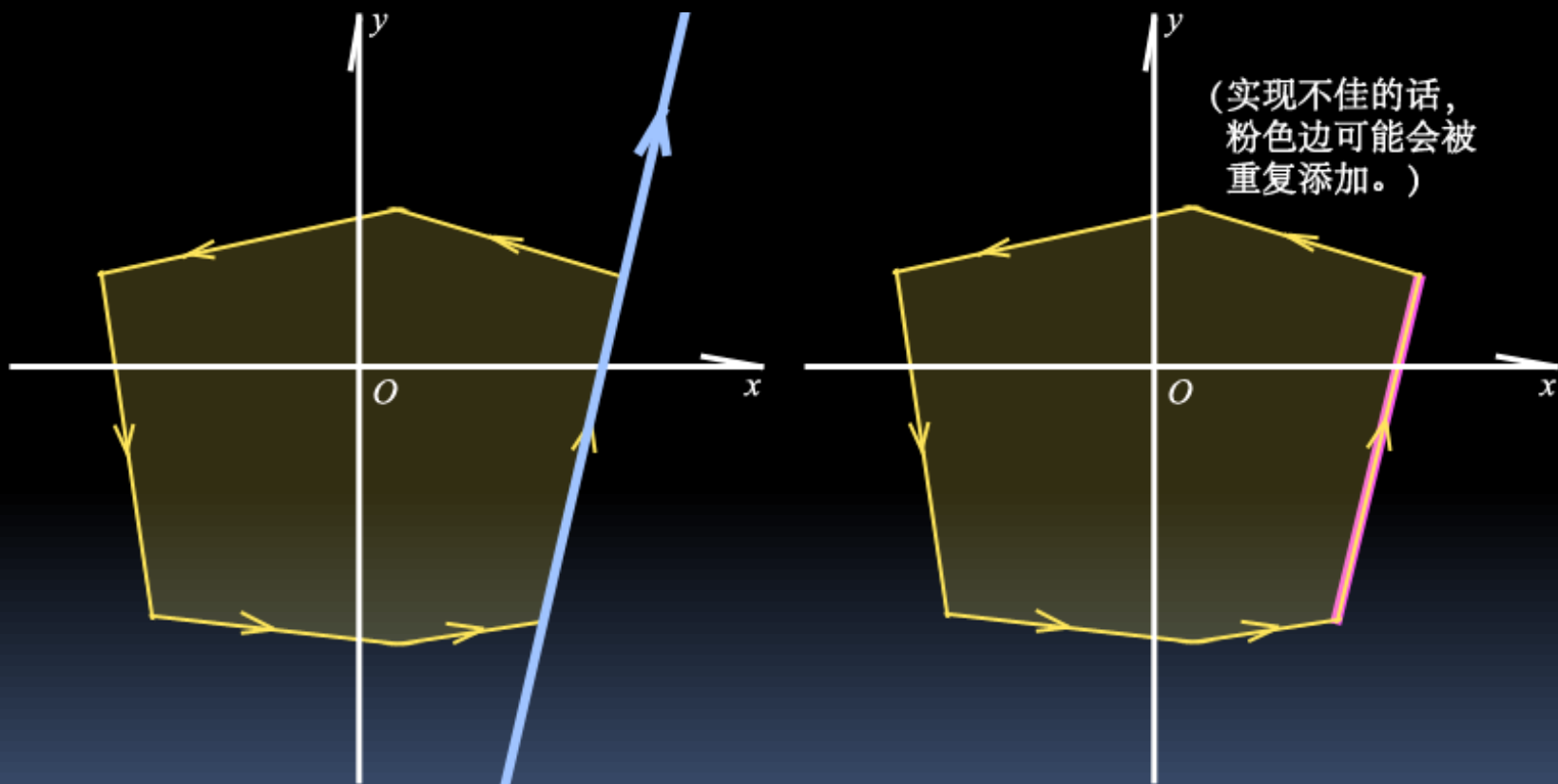
算法演示



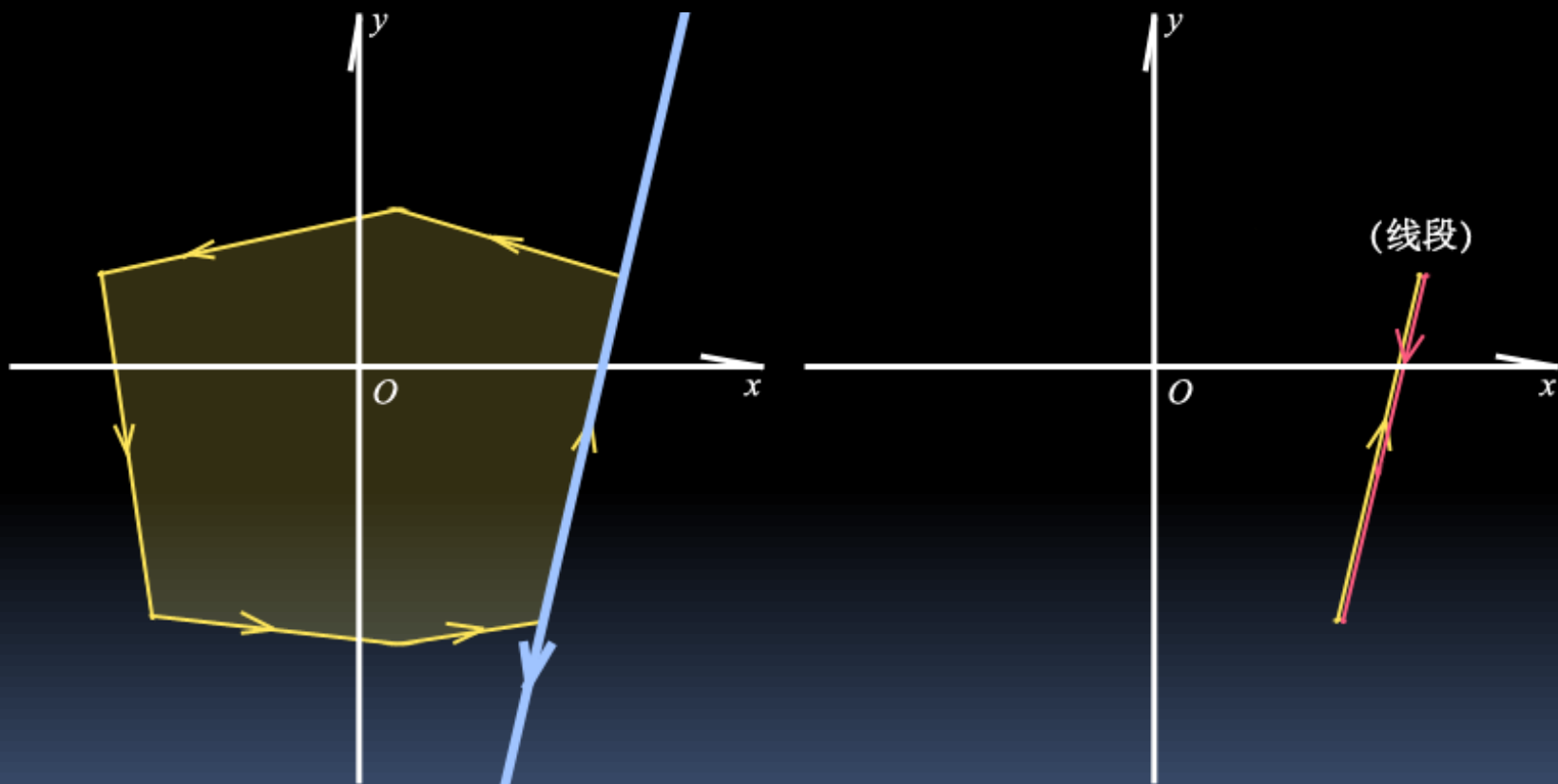
值得注意的边界情况 1



值得注意的边界情况 2



值得注意的边界情况 3

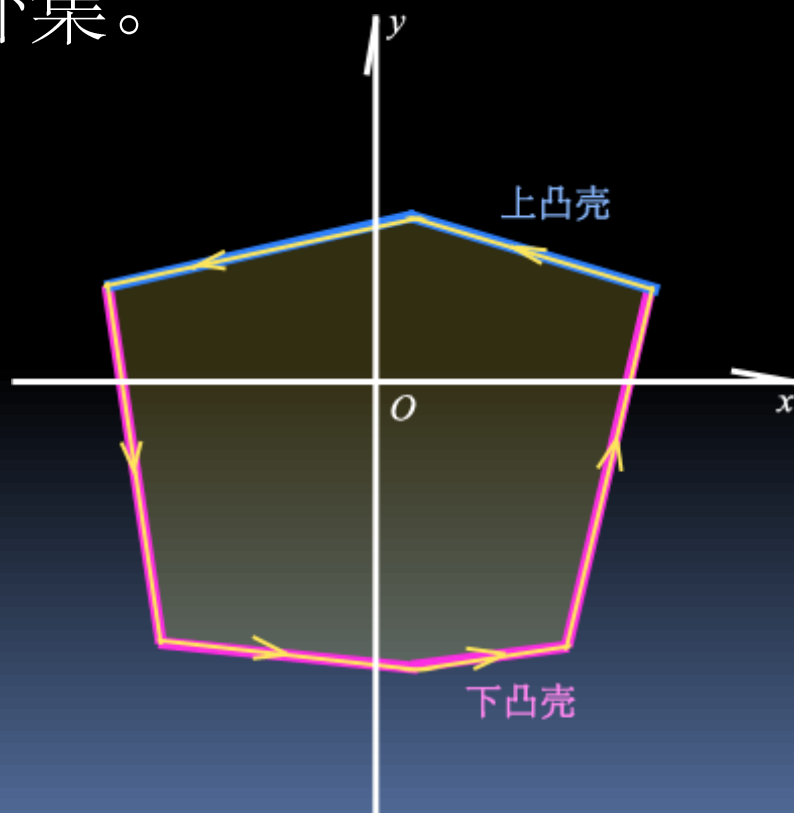


合计 $O(N\log N)$ 的算法

- 这是朱泽园在其国家集训队论文《半平面交的新算法及其实用价值》中提出的算法。
- 由于建立了非 $\sim$ 常 $\sim$ 大的边框，半平面交的结果一定是一个凸多边形。
- 而凸多边形分为下凸壳和上凸壳。如果分开处理这两部分，就会获得非常好的性质。

下凸壳与上凸壳

- 下凸壳中矢量的极角在 $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$ 之间。
- 上凸壳极角取值为其补集。



分别排序

- 按照极角，筛选出可能属于下凸壳和上凸壳的半平面分别按极角大小从小到大排序。
- 上凸壳排序时，极角位于第三象限的半平面要加上 2π .





扫清障碍

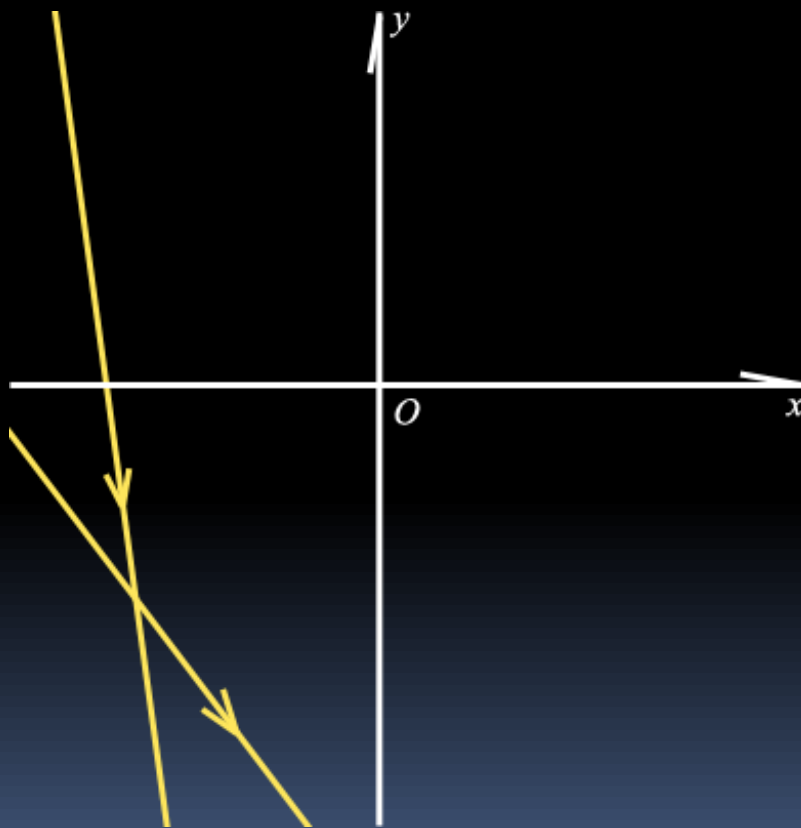
- 对于极角相同的半平面，选择最靠左的一个，并将剩下的舍弃。
 - 这样即保证所有的半平面的极角皆不相同。
- 

处理下凸壳

- 准备一个空栈，并将前两个半平面压入栈。
- 扫描排好序的半平面。对于每个半平面，只要栈中还有两个半平面，即检查其边界直线与栈顶边界直线的交点的横坐标 (记为 x_1) 与栈中前两个边界直线的交点的横坐标 (记为 x_0)。若 $x_0 \geq x_1$ ，则弹出栈顶。重复此过程。
- 若上一步没有弹栈，则将当前半平面压入栈。
- 扫描完成之后栈中即为组成下凸壳的半平面。

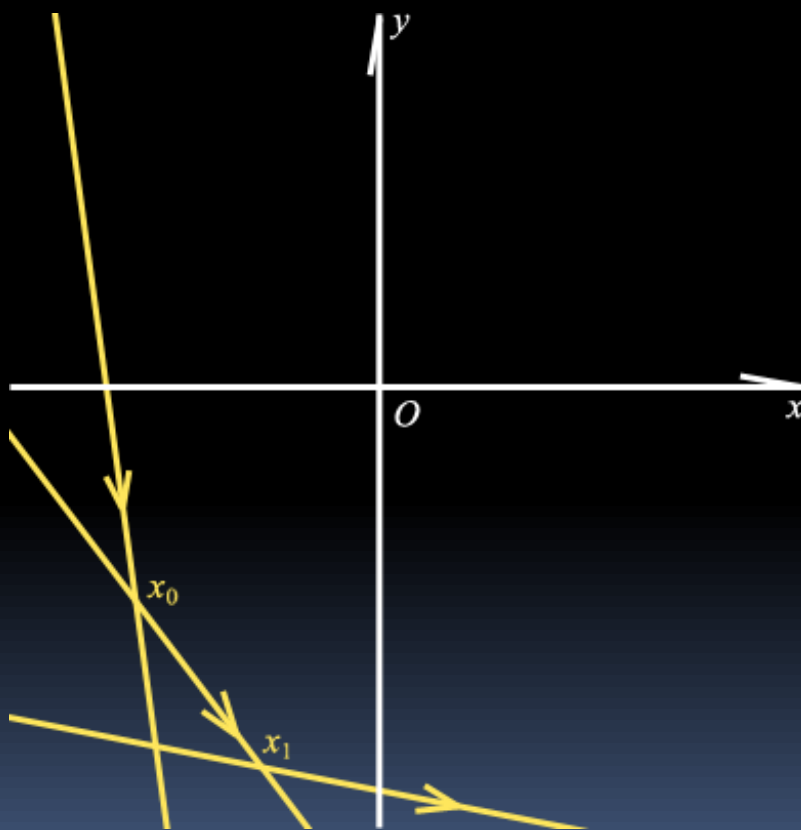
算法演示

首先将前两个半平面进栈。



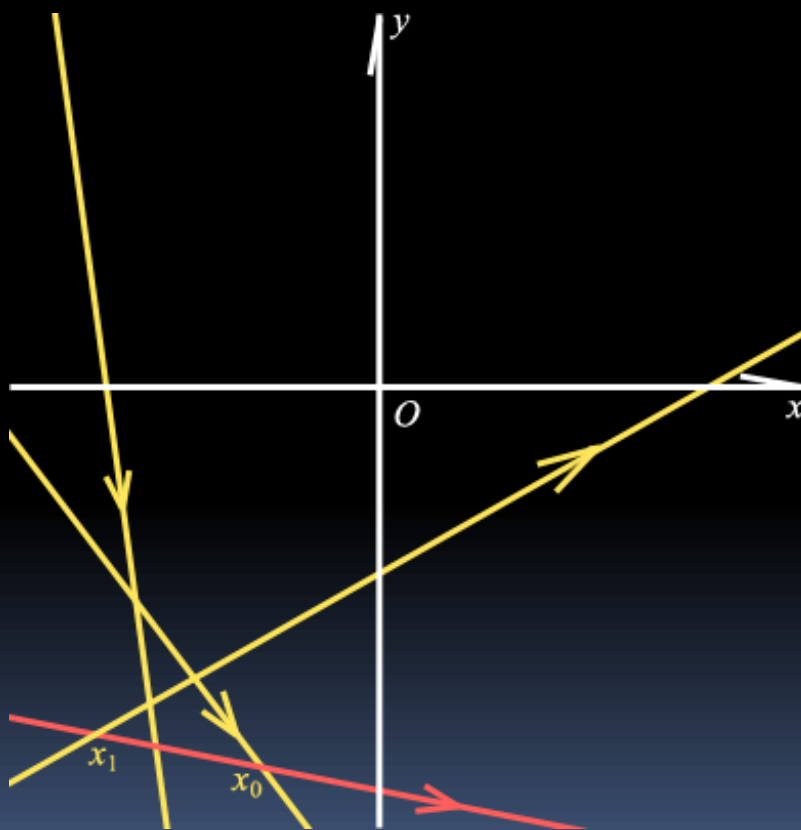
算法演示

$x_0 < x_1$, 将新的半平面压入栈。



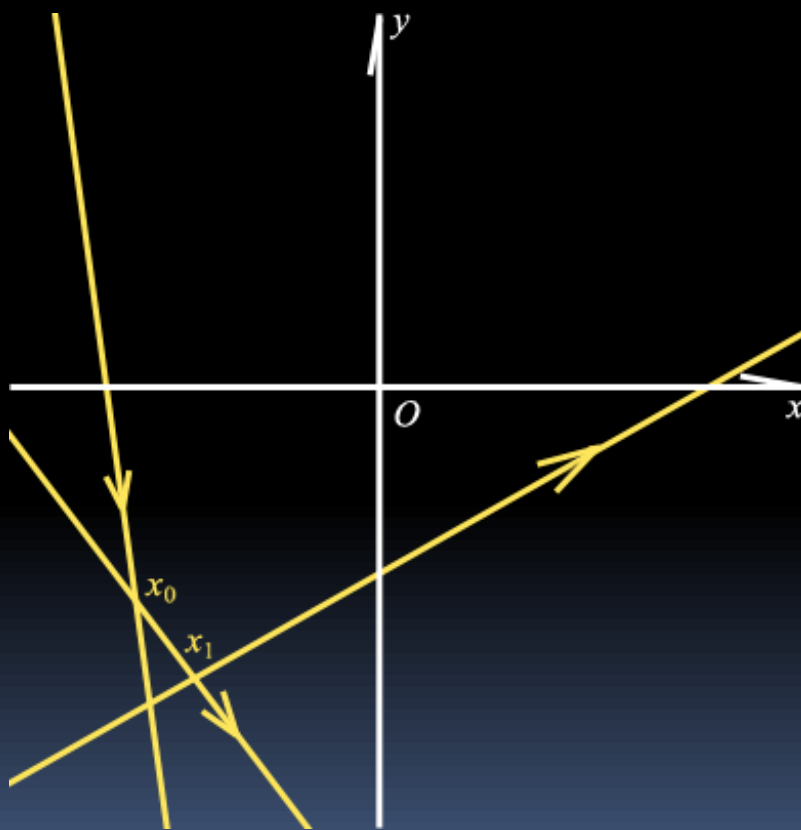
算法演示

$x_0 \geq x_1$ ，将栈顶弹出。



算法演示

$x_0 < x_1$, 将新的半平面压入栈。依此类推。

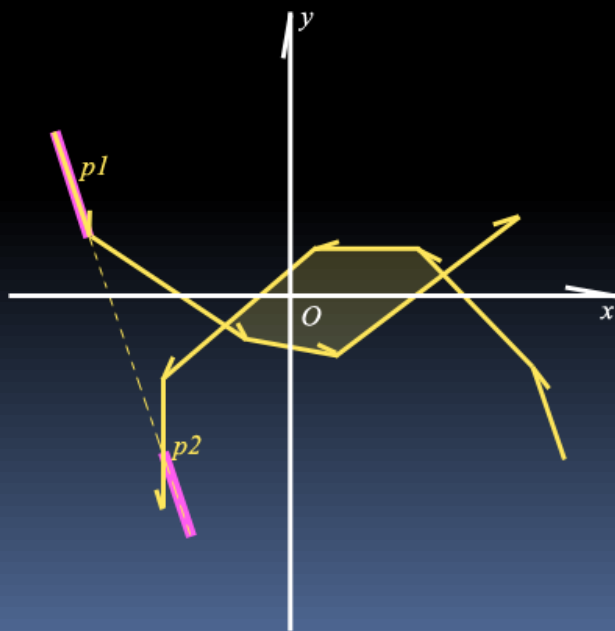


处理上凸壳

- 处理上凸壳的方式完全类似。
- 只是弹栈的条件变为 $x_0 \leq x_1$.

上下凸壳的合并

- 只要凸壳中有不止一个半平面，就有可能出现没有用的半平面。
- 如图，下图中的 $p1$ 半平面没有用。



滤除没有用的半平面

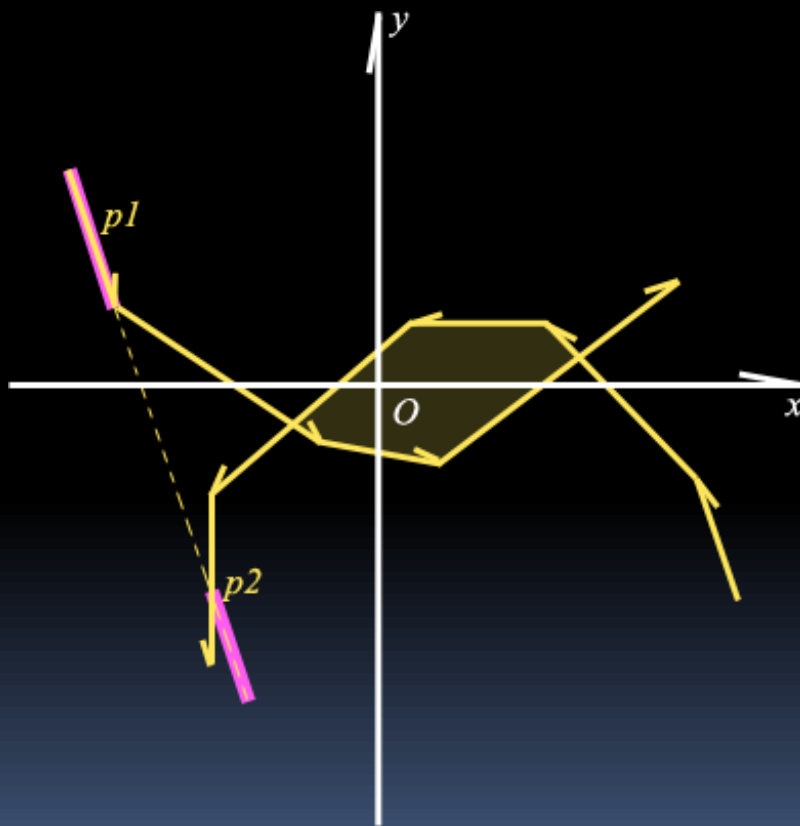
- 先滤除左侧的半平面。指针 $p1$ 指向下凸壳的最左侧， $p2$ 指向上凸壳的最左侧。
- 若下凸壳中还有至少两个半平面，检查 $p2$ 与 $p1$ 的边界的交，以及下凸壳中 $p1$ 右侧的半平面与 $p1$ 的边界的交。若这两个交的并集为空或只有一个端点，则舍弃半平面 $p1$ ，将指针 $p1$ 右移。
- 处理完 $p1$ 后类似处理 $p2$ 。若 $p2$ 右移了，就重复处理 $p1$ ，依此类推，直到二者都不右移了为止。
- 类似滤除右侧没有用的半平面。

实际上……

- “这两个交的并集为空或只有一个端点”仍然只是讨论交点的坐标大小。
- 但是，这两个交点的 x 坐标可能相同，但 y 坐标不相同。所以不能像前面一样只讨论 x 坐标。

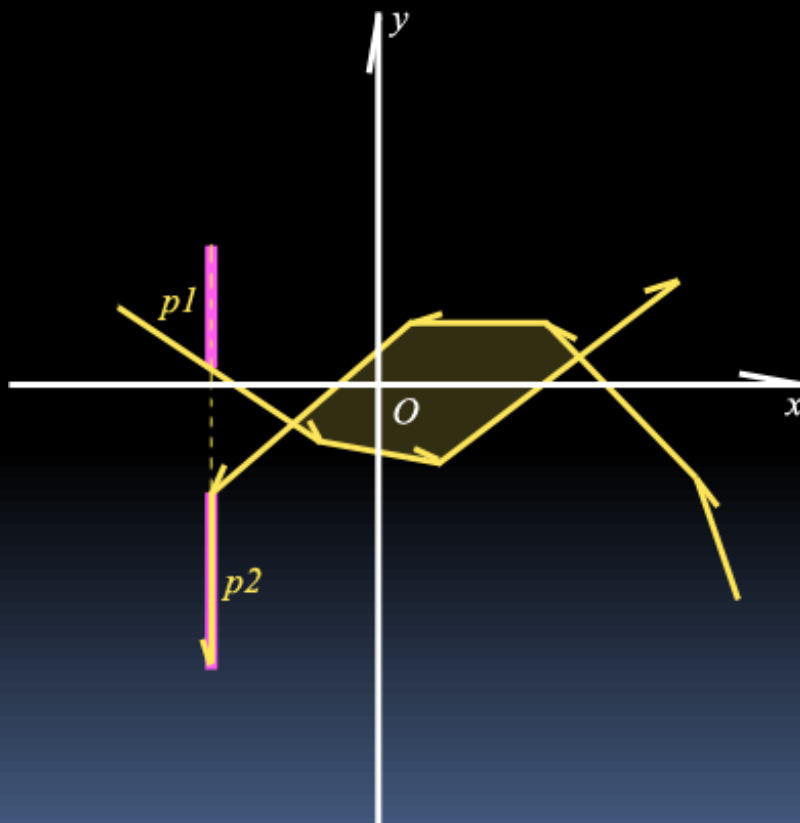
算法演示

$p1$ 上二者的交为空集。右移 $p1$ 。



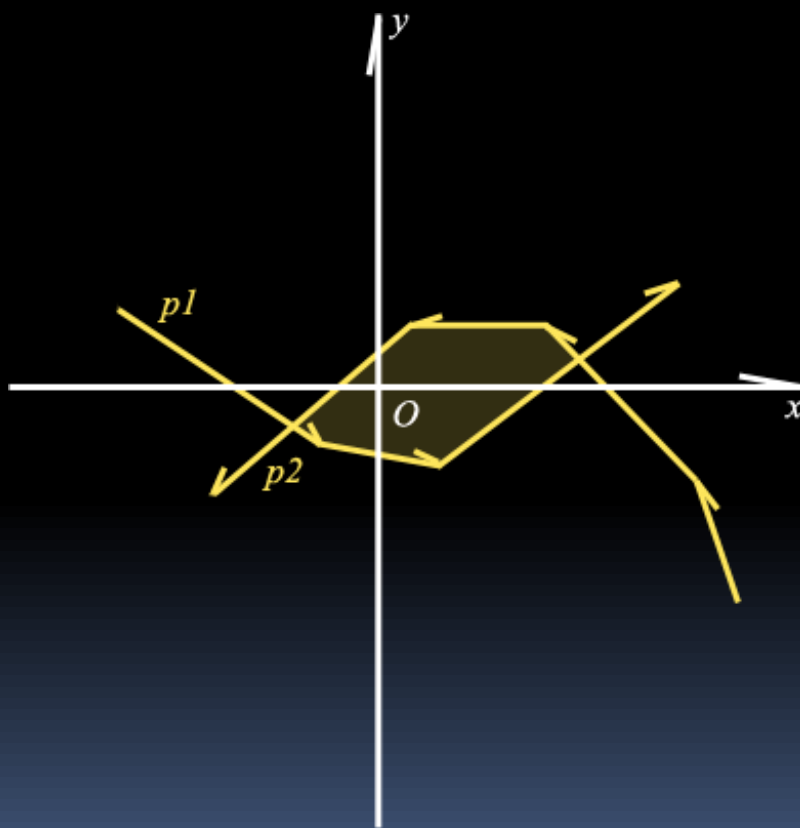
算法演示

$p2$ 上二者的交为空集。右移 $p2$ ，重新检查 $p1$ 。



算法演示

$p1$ 和 $p2$ 均满足要求，结束对左侧的检查。



总结

- 算法的瓶颈仍然在于排序，所以时间复杂度为 $O(N \log N)$ 。若使用桶排序，甚至能得到 $O(N)$ 的半平面交算法，但是并不会会有明显的性能提升。
- 总体来说讨论比较辛苦，写起来不算很难。
- 精度存在比较大的问题，而且半平面交的题目一般都容许 $O(N^2)$ 算法，所以并不是很常用。

半平面交练习题

- POJ 1279 求多边形的核。核中的点满足其到多边形边界上任一点之间的线段只与多边形在那一点相交。
- POJ 3525 求凸多边形最大内切圆半径。
- POJ 2451 裸的半平面交。朱泽园专门为他的算法出的题，要求 $O(N \log N)$ 算法。
- POJ 3384 求在凸多边形内部放两个半径为 r 的圆所能覆盖的最大面积。两圆可以重叠，但不能与多边形相交。需要用到旋转卡壳。

圆

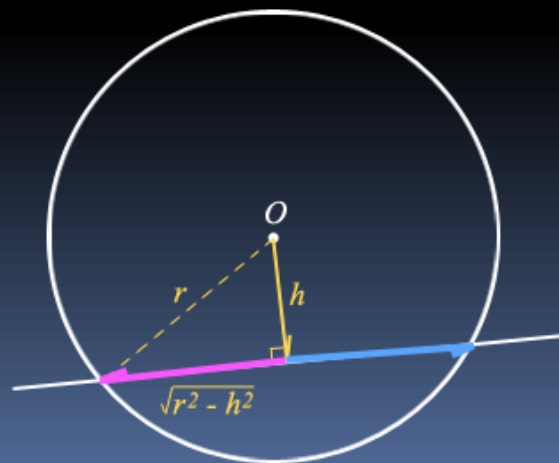
- 到定点 O 距离等于定值 r 的点的集合。
- 有时根据上下文也包含内部的点。
- O 即为圆心， r 即为半径。
- 圆只需要用这两个东西表示就行了。

圆与直线的交点

- 先求圆心到直线的距离 h 。若 $h > r$ 则圆与直线木有交点。
- 否则求圆心到直线的垂直矢量 $\mathbf{H}$ (见第 28 页之定义)。若 $h = r$ 则 $O + \mathbf{H}$ 即为切点。
- 否则……

圆与直线的交点

- 否则记 $H = (x, y)$, 且 $\mathbf{H}$ 逆时针旋转 $\frac{\pi}{2}$ 为 $\mathbf{H}_0 = (-y, x)$ 。
- 记 $d = \sqrt{r^2 - h^2}$ 为垂足到交点的距离, 则两个交点分别为 $O + \mathbf{H} + \frac{d\mathbf{H}_0}{h}$, $O + \mathbf{H} - \frac{d\mathbf{H}_0}{h}$ 。
- $\uparrow$ 注意 $|\mathbf{H}_0| = h$ 。

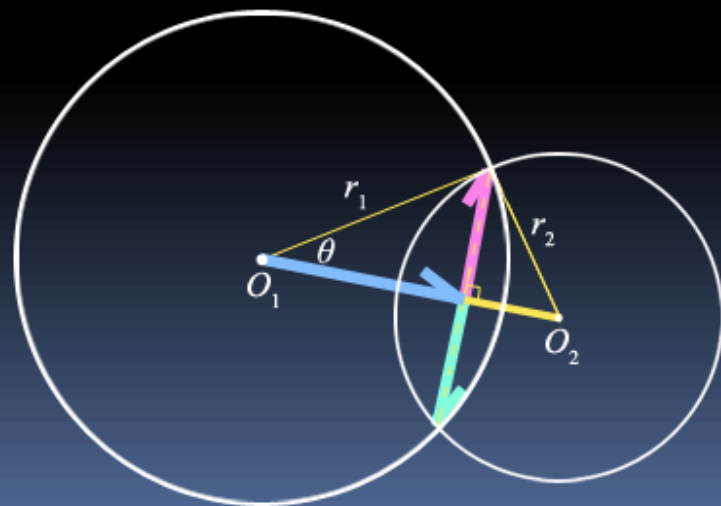


圆与圆的交点

- 先求二圆的圆心距 d ，并记二圆的半径为 r_1 和 r_2 。若 $d > r_1 + r_2$ ，则二圆外离；若 $d < |r_1 - r_2|$ ，则二圆内含。这些情况下都不会有交点。
- 上面不等式若取等号，则二圆分别为外切和内切。处理方式见下页。
- 否则……

圆与圆的交点

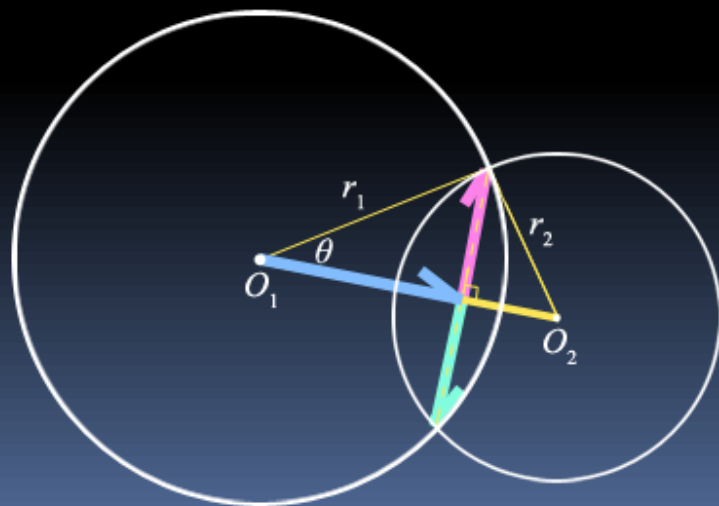
- 如图所示, 据余弦定理有 $\cos \theta = \frac{r_1^2 + d^2 - r_2^2}{2r_1 d}$.
- 设 $\mathbf{D} = \mathbf{O}_2 - \mathbf{O}_1$. 若二圆外切或内切, 则 $\mathbf{O}_1 + \frac{\mathbf{D}r_1 \cos \theta}{d}$ 则为唯一交点。
- 否则……



圆与圆的交点

- 设 $\mathbf{D}$ 逆时针旋转 $\frac{\pi}{2}$ 为 $\mathbf{D}_0$.
- 而 $\sin \theta = \sqrt{1 - \cos^2 \theta}$, 故两个交点分别为

$$O_1 + \frac{\mathbf{D}r_1 \cos \theta}{d} + \frac{\mathbf{D}_0 r_1 \sin \theta}{d}, \quad O_1 + \frac{\mathbf{D}r_1 \cos \theta}{d} - \frac{\mathbf{D}_0 r_1 \sin \theta}{d}.$$

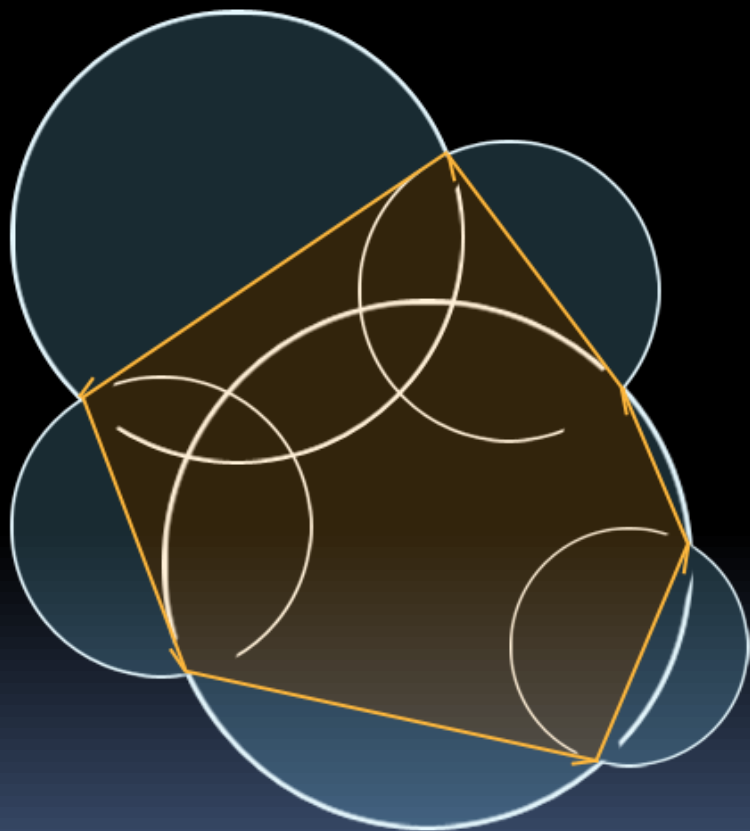


圆的面积并

- 题目: [SPOJ CIRU](#)
- 给出 N ($N \leq 1000$) 个圆, 求其并集的面积。
- 要求 $O(N^2 \log N)$ 的算法。
- 后面给出的算法由 AekdyCoin 原创。

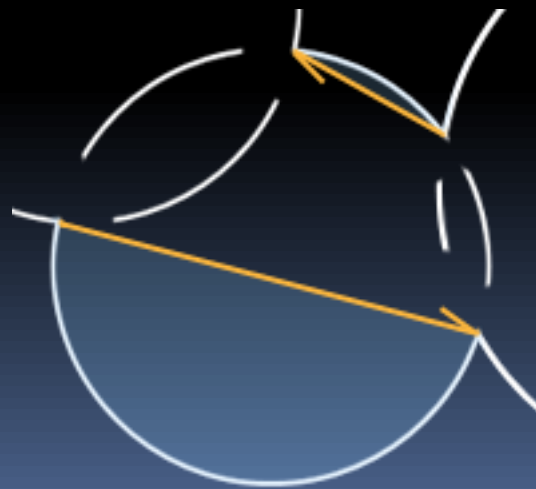
分析

- 如图，最后形成的图形是由很多弓形和多边形组成的。
- 弓形可以单独计算，多边形利用叉积亦可以分各边计算，所以我们可以单独考虑每个圆。



每个圆的情形

- 如图，对于每个圆，用其他的圆与之求交，找到所有没有被覆盖过的圆弧。圆弧须以逆时针序表示。
- 对于每条圆弧，累加其弓形的面积，以及这条边的有向面积。
- 半径为 r ，角度为 θ 的弓形面积为。

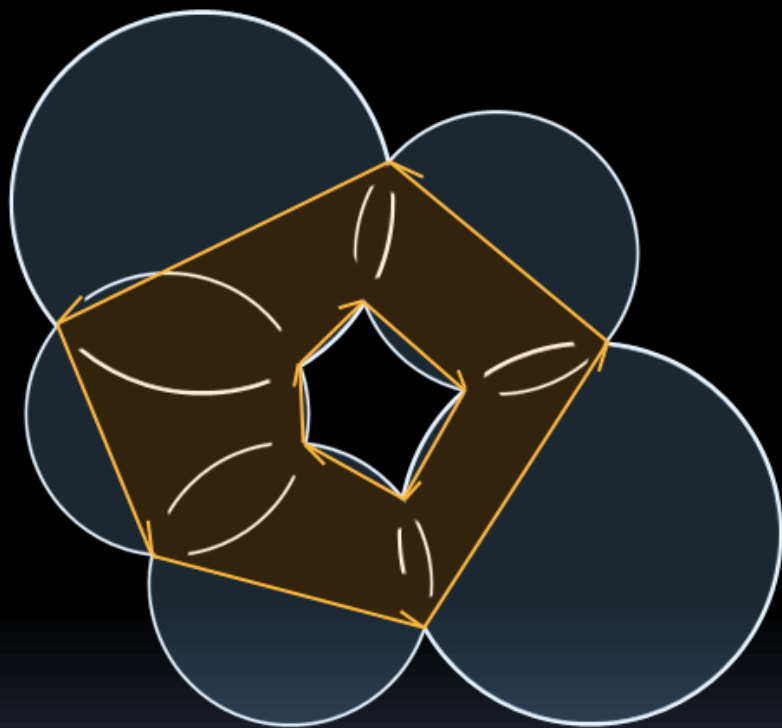


不连通?

- 由于各个连通块中的弓形和多边形的边实际上都得到了很好的处理，所以图形不连通没有问题。

有“洞”？

- 发生了非常 NB 的事情。由于我们累加的是有向面积，而“洞”中多边形是反向（顺时针）旋转的，所以“洞”会被自动抵消！



时间复杂度

- 对于每个圆，要求 $O(N)$ 次交，从而出现了 $O(N)$ 个交点。要找出未被覆盖的弧的话，要对交点进行排序，所以每个圆要花费 $O(N \log N)$ 的时间进行处理。
- 总时间复杂度 $O(N^2 \log N)$ 。

算法扩展

- **SPOJ CIRUT** 求恰好被覆盖 1 次、2 次、 $\dots$ 、 N 次的面积。
- 算法的本质在于对“裸露”部分进行处理。这种方法也可以用于求很多圆与很多凸多边形的面积并——但是情况复杂得多，要讨论的情况也多得多。



三维矢量



第三维度

- 三维矢量用 (x, y, z) 表示。
- 点积和标量乘法天然是在多维上定义的概念，所以在三维空间中可以立即使用。
- 矢量的模，夹角，垂直，缩放，投影，对称等扩展概念和计算方法也立刻投入了我们的怀抱。

三维叉积

- 但是二维叉积却不适用了。在此我们要定义三维叉积。
- 三维叉积的结果是三维矢量。
- 我们有 .
- 本质是行列式，原式，其中 .

左手系与右手系

- x, y, z 轴的方向，传统上应该满足右手系。
- 即伸出右手，将四指由 x 轴指向沿小于平角转到 y 轴指向，拇指所指即为 z 轴指向。
- 类似也可以定义左手系。三个不共面的矢量必然满足左手系与右手系中的一种。



文字游戏

- 注意 “ a, b, c 成右手系” 和 “ b, a, c 成右手系” 是截然相反的两个命题。
- 但是 “ a, b, c 成右手系” 和 “ b, c, a 成右手系” 是同一个命题。
- 画图看一下这是为什么。

三维叉积的性质

- 若 $\mathbf{a} \times \mathbf{b} = \mathbf{c}$ ，则 $\mathbf{a} \cdot \mathbf{c} = \mathbf{b} \cdot \mathbf{c} = 0$ ，即叉积的结果与原矢量垂直。
- $\mathbf{a}$, $\mathbf{b}$ 和 $\mathbf{a} \times \mathbf{b}$ 应满足右手系。即伸出右手，将四指由 $\mathbf{a}$ 沿小于平角转到 $\mathbf{b}$ ，拇指所指的方向即为 $\mathbf{a} \times \mathbf{b}$ 的方向。
- $|\mathbf{a} \times \mathbf{b}|$ 的值在数值上等于 $\mathbf{a}$ 和 $\mathbf{b}$ 所夹的平行四边形的面积。

在三维空间我们失去的

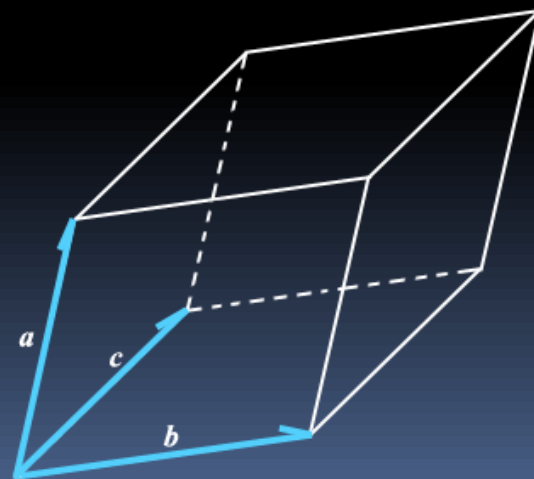
- **极角**。在三维空间中描述矢量的方向不能只用一个角度，实际上需要使用两个角度。
- **左右**。在三维空间中，**两个矢量间**没有左与右的概念。取而代之的是**三个矢量间****左手系**与**右手系**的概念。

混合积

- $(\mathbf{a} \times \mathbf{b}) \cdot \mathbf{c}$ 称为矢量 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 的混合积。
- 又记为 $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ 。
- 当矢量 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 成右手系时，混合积的值为正。
- 但是这到底是个什么玩意儿？

有向体积

- 如图。 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 的混合积的模，即为 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 所成的平行六面体的体积。
- 考虑符号的话，混合积又称为有向体积。
- $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 所成三棱锥的体积为平行六面体的 $1/6$ 。



一切的本质：行列式

- 若 $\begin{vmatrix} a & b & c \\ b & c & a \\ c & a & b \end{vmatrix} > 0$ ，那么
- 由此我们可以得到左右手系的新定义：若上面行列式为正，则 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 成右手系，否则成左手系。
- 这个结论和定义可以扩展到高维！



三维计算几何

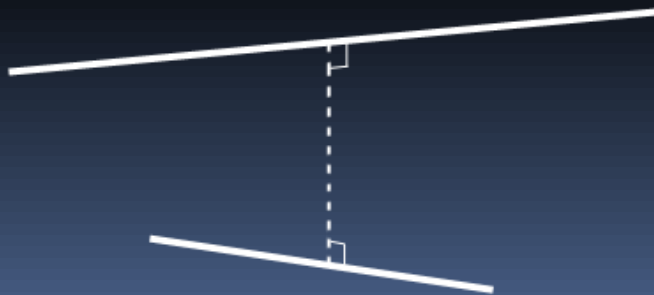


三维直线

- 三维直线的表示法与二维类似。
- 计算点到直线的距离的方法与二维相同。
- 计算分点的方式也与二维相同。

异面直线

- 但是在三维空间中，直线除平行与相交之外有着第三种位置关系：**异面**。
- 异面直线的距离即为两条直线上距离最小的两点之距离。你们都知道这就是两条直线的**公垂线段**的长度。

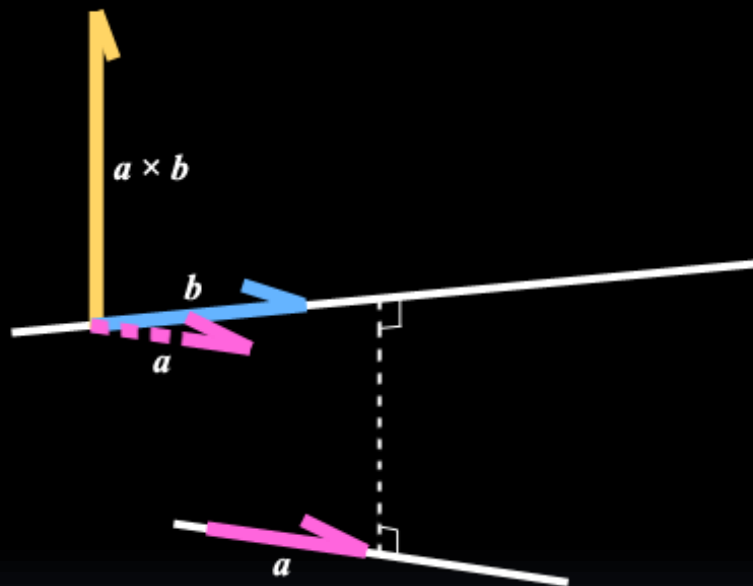


异面直线的距离

- 设两条直线的方向矢量分别为 $\mathbf{a}$ 和 $\mathbf{b}$ 。若 $\mathbf{a} \times \mathbf{b} = \mathbf{0}$ ，则两直线平行，它们一定共面，其距离为 0.
- 否则……

异面直线的距离

- 否则两条直线上各任取一点 A 和 B 。
- 矢量 $B - A$ 在 $a \times b$ 上的投影的模即为所求。



两直线的交点

- 先求它们的距离。若不为 0，显然木有交点。
- 否则，若它们的方向矢量平行，它们重合。
- 否则，它们是相交的。推荐利用后面所讲的**投影技术**将其转化为二维问题。



平面

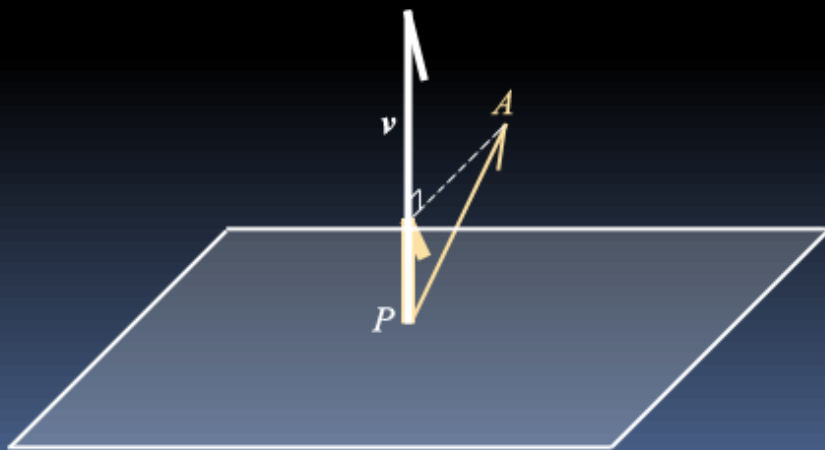
- 平面是到两点距离相等的点的轨迹。
 - 具体是个啥乃们应该清楚……
- 

点法式

- 与平面垂直，即与平面上一切矢量垂直的非零矢量叫做平面的法向量。可以证明平面的所有法向量平行。
- 记录平面的一个法向量 $\mathbf{v}$ ，再记录平面上一点 P ，就可以完整表示整个平面了。

点到平面的距离

- 点 A 到平面的距离，即为 $A - P$ 在 $\mathbf{v}$ 上的投影长度。
- $P - A$ 在 $\mathbf{v}$ 上的投影我们也称做垂直矢量。注意，我自己起的名字。

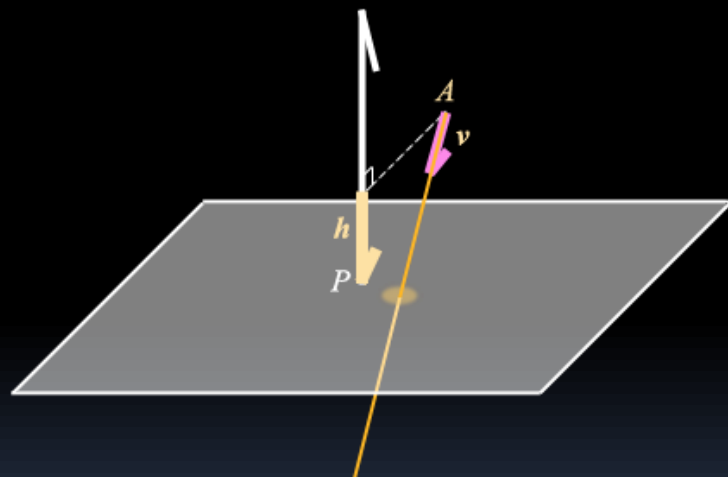


直线与平面的交点

- 设直线经过点 A ，方向矢量为 $\mathbf{v}$ 。记 A 到平面的垂直矢量为 $\mathbf{h}$ 。
- 若 $\mathbf{v} \cdot \mathbf{h} = 0$ ，则直线与平面平行。直线可能落在平面上，也可能与平面没有重合部分。
- 否则……

直线与平面的交点

- 目的是通过缩放 $\mathbf{v}$ 得到 $\mathbf{v}_0$, 使得 $\mathbf{v}_0$ 到 $\mathbf{h}$ 的投影等于 $\mathbf{h}$ 。即
$$\mathbf{v}_0 \cdot \mathbf{h} = h^2.$$
- 故 $\mathbf{v}_0 = \mathbf{v} \frac{h^2}{\mathbf{v} \cdot \mathbf{h}}$,
 $A + \mathbf{v}_0$ 即为所求交点。



平面与平面的交线

- 设两个平面的法矢量为 $\mathbf{h}_1, \mathbf{h}_2$. 若它们平行, 则两平面平行。它们可能重合, 或者没有公共点。
- 否则, 交线的方向矢量与 $\mathbf{h}_1, \mathbf{h}_2$ 垂直, 即为 $\mathbf{h}_1 \times \mathbf{h}_2$. 剩下的工作就是求交线上一点。
- 可以随机生成一条在第一个平面上的, 且不与第二个平面平行的直线与第二个平面求交来确定此点。



二面角

- 就是两个平面的法矢量的夹角啦。



平面上点的二维投影

- 有时一些几何对象出现在同一个平面上。我们希望给予平面上每一个点一个二维坐标，从而把问题转化为一个二维问题加以解决。
- 我们需要在平面上指定一个点 O 作为原点，以及一个矢量 $\mathbf{x}$ 作为横轴。即 $\mathbf{v} \cdot \mathbf{x} = 0$.

平面上点的二维投影

- 此时，取 $\mathbf{y} = \mathbf{v} \times \mathbf{x}$ 作为纵轴，即可保证 $\mathbf{x}$, $\mathbf{y}$, $\mathbf{v}$ 成右手系。
- 对于平面上的每一个点 A ，在这套坐标系下其横坐标即为 $A - O$ 在 $\mathbf{x}$ 上的投影长度

$$\frac{(A-O) \cdot \mathbf{x}}{|\mathbf{x}|}, \text{ 纵坐标亦然。}$$

平面上点的二维投影

- 为了避免每次计算 $|\mathbf{x}|, |\mathbf{y}|$, 我们可以将 $\mathbf{x}, \mathbf{y}$ 的模缩放为 1。
- 这样 A 的坐标即为 $((A - O) \cdot \mathbf{x}, (A - O) \cdot \mathbf{y})$.
- 若已知平面上一点以此坐标系投影的坐标 (a, b) , 则其三维坐标即为 $O + a\mathbf{x} + b\mathbf{y}$.

多面体

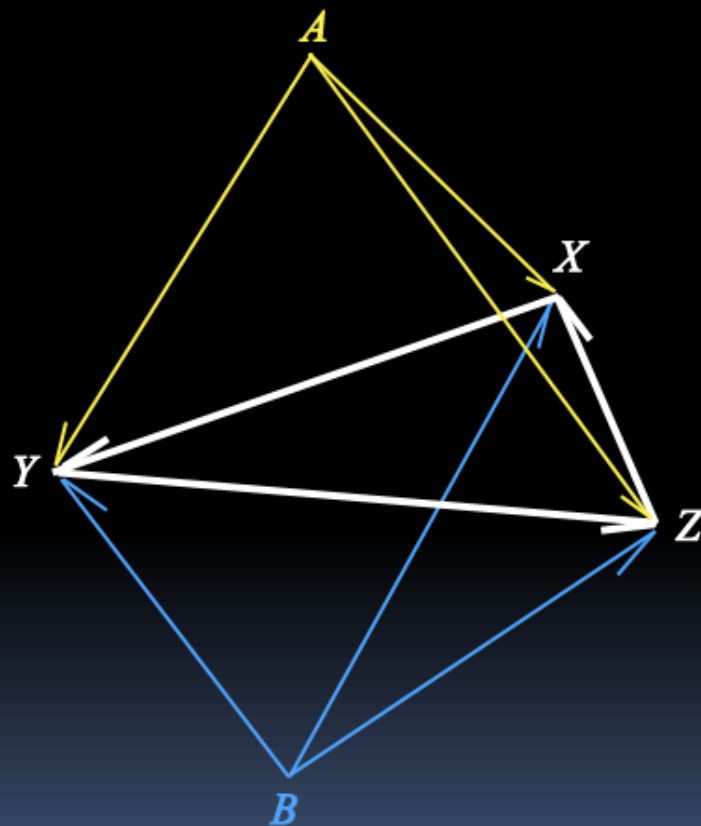
- 多面体是指三维空间中由平面和直边组成的几何形体。
- ……以上是科学定义。我不觉得这么一句废话有什么用。
- 我们假定多面体的每个面都是**三角形**的。

凸多面体

- 若一个多面体是凸集，则它是凸多面体。
- 如同凸多边形是半平面的交一样，凸多面体也可以看做半空间的交。
- 求半空间的交非常复杂，在此不多做介绍。不过有关于半空间的基本知识仍要加以讲解。

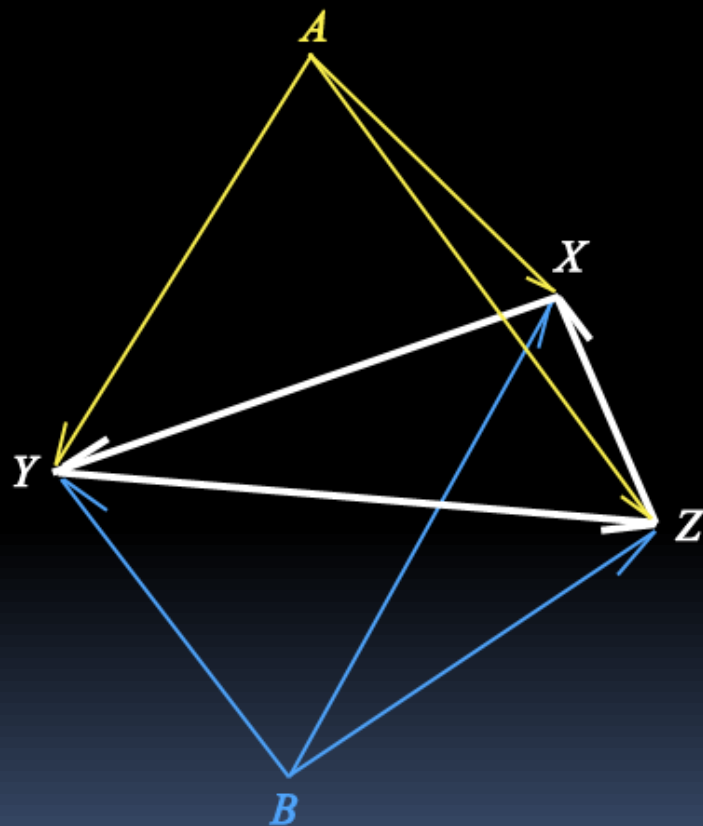
半空间的表示

- 如同半平面可以用两个互异的有序点来表示一样，半空间可以用三个不共线的有序点来表示。



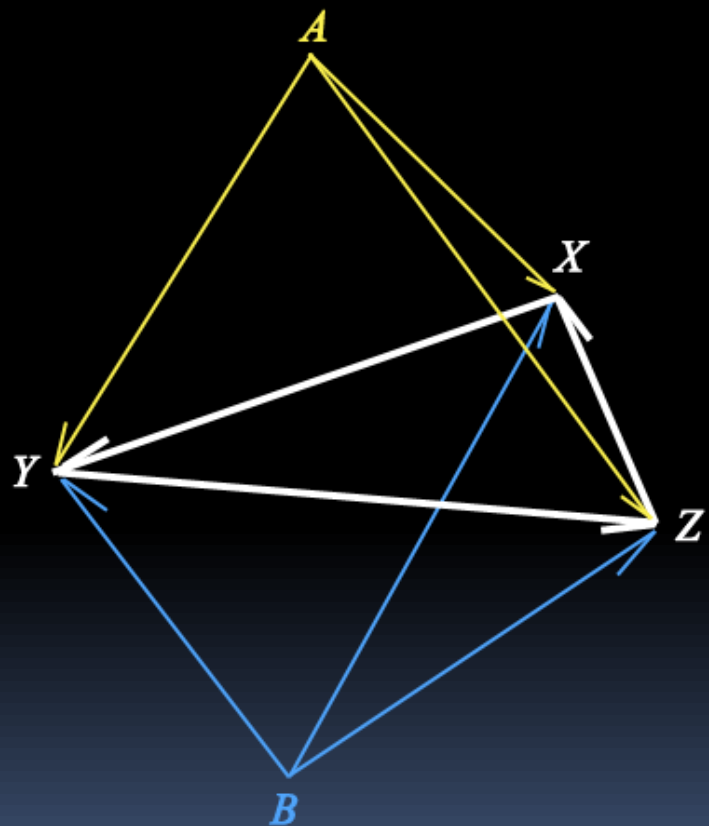
半空间的表示

- 如图，在点 A 的一侧，矢量 $\overrightarrow{AX}, \overrightarrow{AY}, \overrightarrow{AZ}$ 成左手系，我们称这一侧为半空间的**外侧**。
- 类似地，在 B 的一侧三个矢量成右手系，这一侧即为半空间的**内侧**。



简单判别法

- 若要判断点 P 在内侧还是外侧，则从点 P 向平面看去。
- 若半空间的三个有序点成逆时针，则点 P 在**外侧**；否则点 P 就在**内侧**。



凸多面体的表示

- 表示一个凸多面体需要记录其所有的面。对于所有的面，要正确记录这个面上点的顺序。
- 根据假设，每个面都是三角形的，即包含三个点，所以每个面都表示了一个半空间。
- 凸多面体应为这些半空间**内侧**的交。

可视

- 若 P 在半空间 A 的外侧，则称半空间 A ，或 A 所代表的平面对 P 是可视的。
- 我们在观察一个凸多面体的时候，看到的面即为视点的可视面。
- 凸多面体各个面的点的顺序应该保证，从多面体内部的任意一点看去，所有的面都不可视。
- 若将左右手系用行列式的符号替代，则可视的概念能推广到高维。

若多面体有非三角面

- 这样的面记录的时候要保证，其上所有点从多面体的内部看去成**顺时针**排列。
- 此时这个面仍然能有效地表示一个半空间。

三维凸包

- 从凸包的定义出发推断，一坨三维空间中的点的凸包是一个凸多面体。
- 事已至此我们只好想办法求它了。但是不难发现由于失去了左右这一概念，Graham扫描法彻底报废了。
- 那怎么办？
- 为了方便起见，我们假定没有四点共面的情况。

暴力方法

- $O(N^3)$ 枚举所有的点的三元组，将其看做半空间。若所有其它的点都在这个半空间的内侧，则这个面是凸包的一个面。
- 时间复杂度 $O(N^4)$ 。非常好写！

增量法

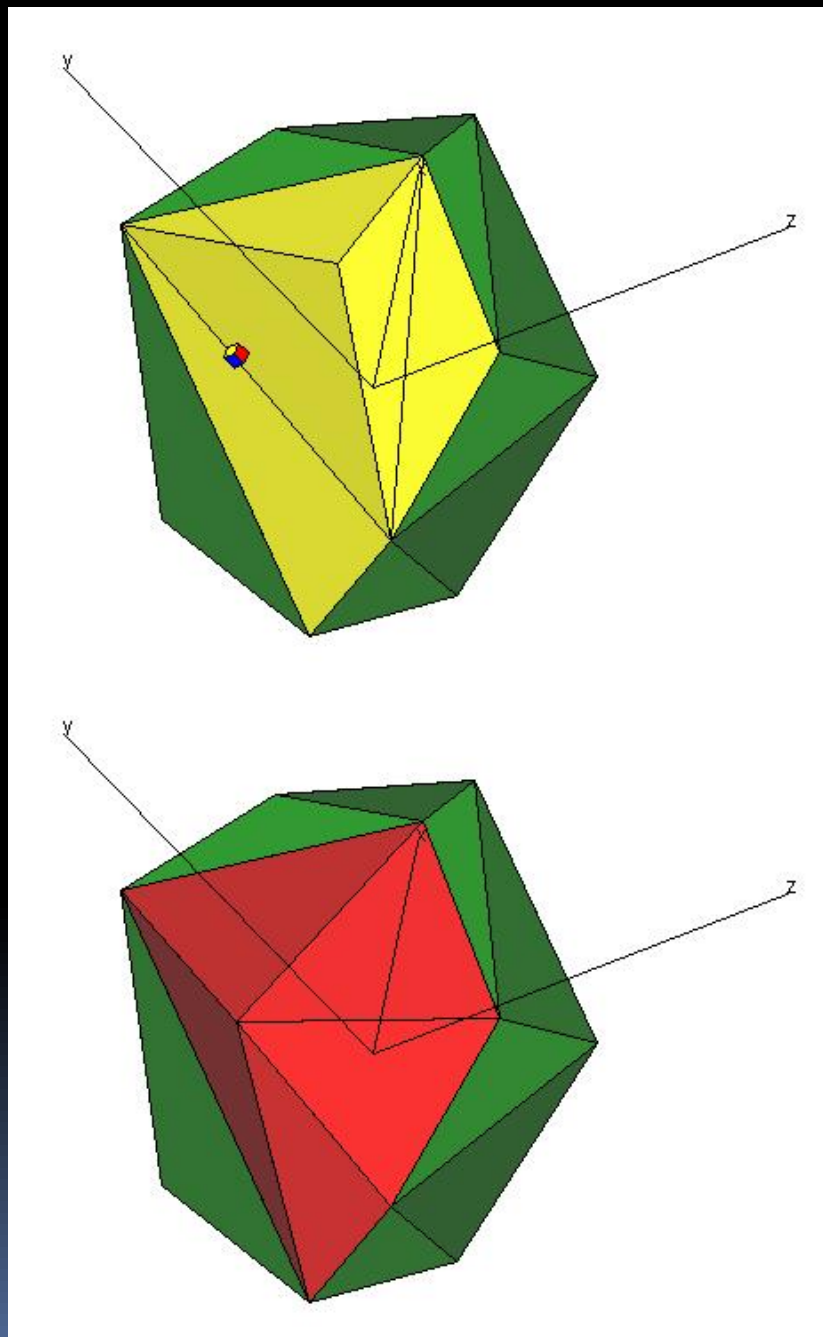
- 增量法作为求凸包的有力算法，可以解决任意维度的凸包问题。
- 最坏情况下其时间复杂度为 $O(N^2)$ 。

算法流程

- 取出点集中的四个点，这四个点组成的四个面都是初始凸包的面。调整各个面上点的顺序使得从内部看去各个面均不可视。
- 之后添加其它各点。如果从这个点看去，所有的面均不可视，则此点在凸包内部。什么都不做，跳过该点。
- 否则……

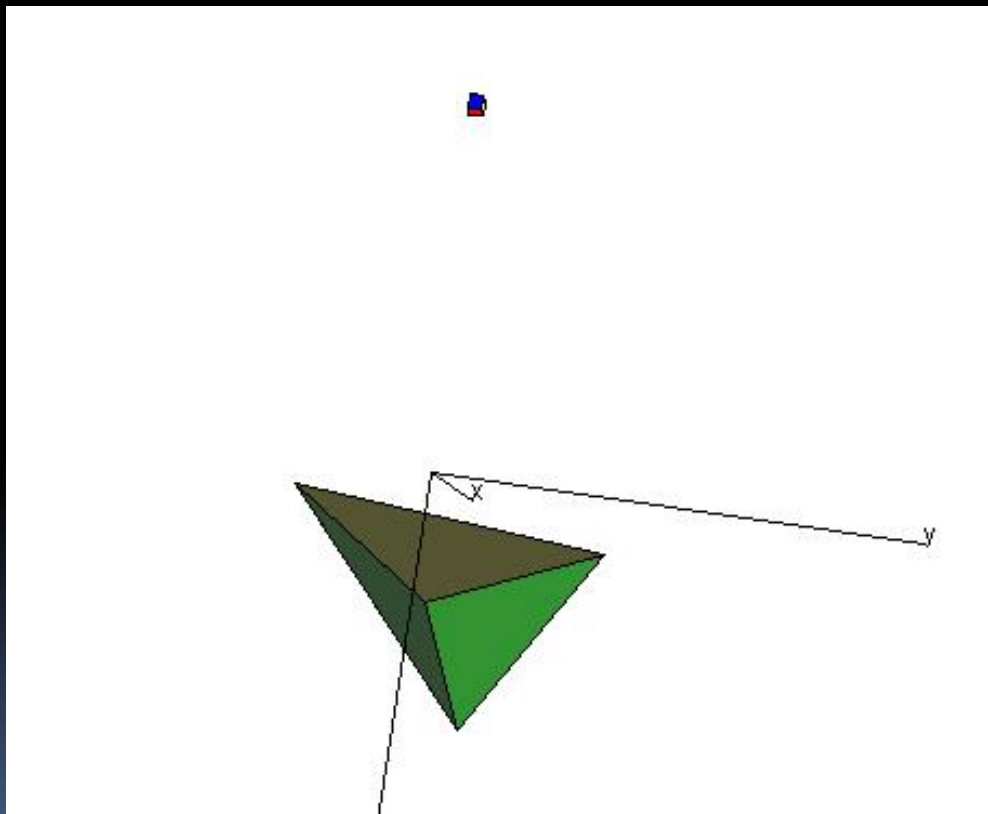
算法流程

- 设当前点是 P 。
- 找到可视面和不可视面的分界线。删除掉所有的可视面，对于不可视境界线上的每条边 $A \rightarrow B$ ，建立过此边和当前点的面 $A \rightarrow B \rightarrow P$ 。



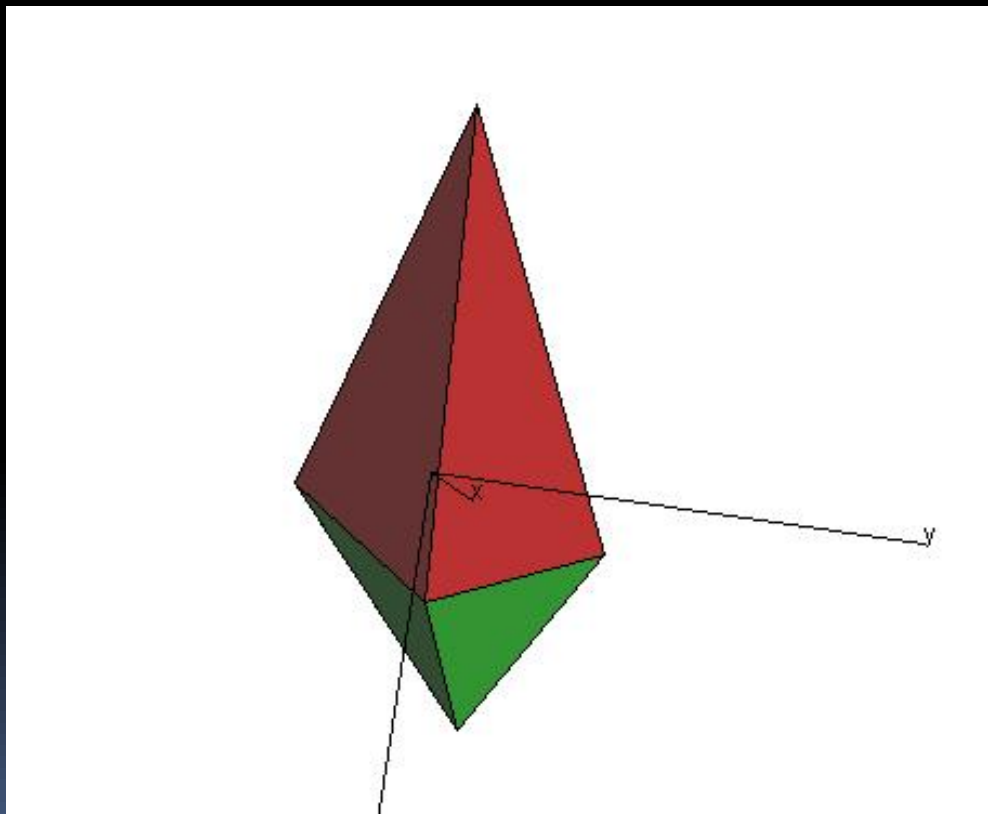
算法演示

建立初始凸包，并试图添加第一个点。



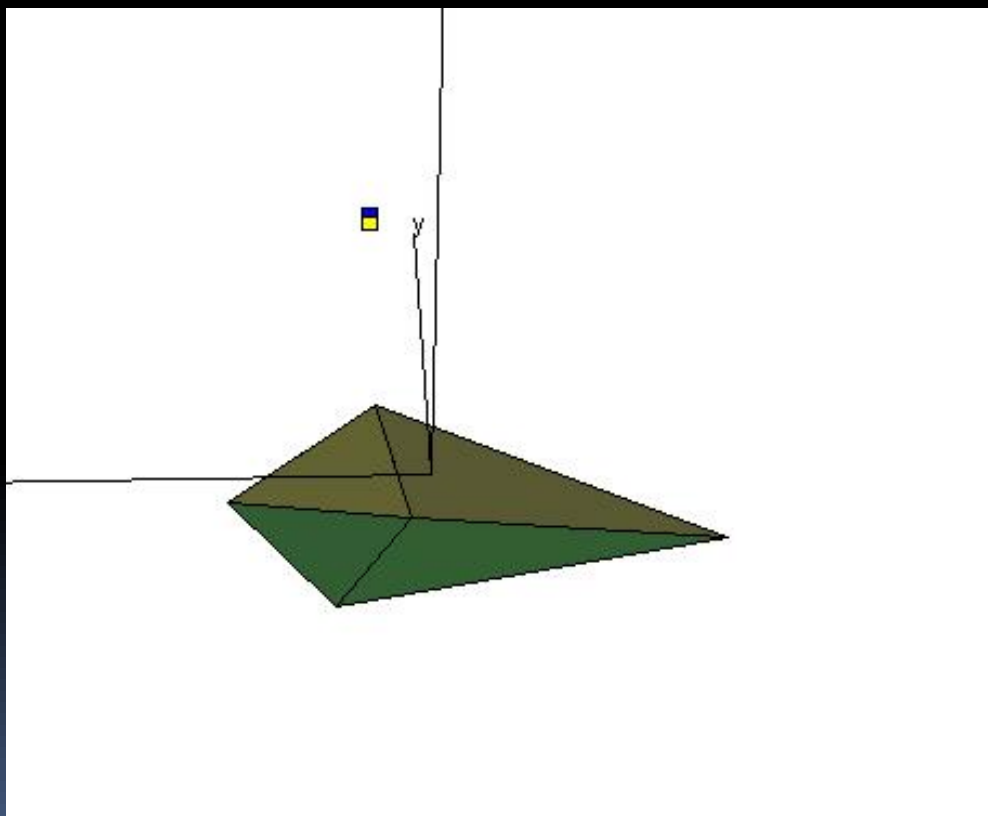
算法演示

添加第一个点完毕。



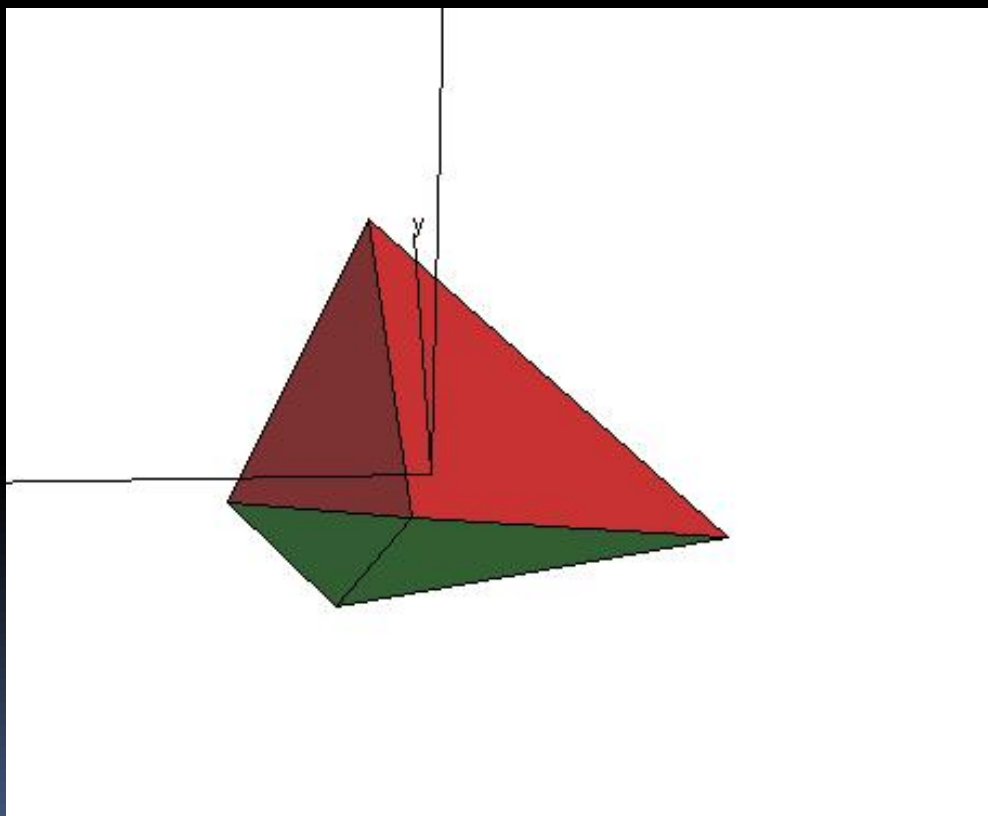
算法演示

试图添加第二个点。



算法演示

添加第二个点完毕。如此添加所有点即可。



随机增量

- 可以证明，在平均情况下算法的期望时间复杂度是 $O(N\sqrt{N})$.
- 所以可以先随即打乱初始点集中点的顺序来得到期望复杂度。
- 这即为随机增量算法。

三维凸包练习题

- POJ 3528 裸的三维凸包，且不含四点共面。
- 三维凸包的出题概率很低，就不提供更多的练习题了。

球

- 在三维空间中，到定点距离等于定值的点的轨迹即为球。
- 球可以用球心 O 和半径 r 来表示。
- 二维球面上的几何学称为球面几何，是非欧几何的一种。自然我们不会研究如此高深的问题。

球与直线的交点

- 首先求球心到直线的距离 h 和垂足。
- 若 $h > r$ ，则直线与球相离。
- 否则在直线上垂足到交点的距离为 $\sqrt{r^2 - h^2}$ ，缩放直线的方向矢量计算这两个点即可。

球与平面或球的交圆

- 既然已经到了如此程度，想必大家已经可以自己来求了吧。
- 先求交圆的圆心，再求交圆的半径。由于这个圆是在三维空间中的，需要标明其在哪个平面上。

经纬度

- 大家参考地理课所学。
- 在球面上，一点的纬度范围为 $[0, \pi]$ ，经度范围为 $[-\pi, \pi]$.
- 北极的纬度是 0 ，赤道的纬度是 $\pi / 2$.
- 本初子午线的经度是 0 .

经纬度到三维坐标的转换

- 若球心在原点，则 z 轴指向北极， x 轴指向本初子午线的话，则球上经度为 ϕ ，纬度为 θ 的一点的三维坐标为

$$O + r(\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta)$$

- 若非如此，需要进行三维旋转。



三维旋转

- 确认旋转之后坐标轴的位置，利用与昨天所讲类似的旋转矩阵求解即可。



计算几何杂题



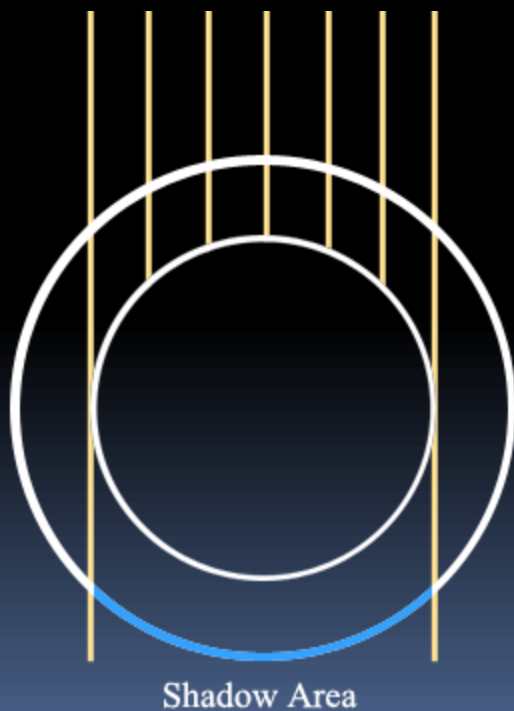
Polaris of Pandora

「潘多拉的北极星」

- HDU 4428 / ZOJ 3663
- 潘多拉星球的半径为 r . 在其北极的正上方有一颗北极星，距离星球可以认为是无穷远 (即星光可以认为是平行光)。
- 现在要从经纬度为 $(lat0, lng0)$ 的点沿大圆飞行到经纬度为 $(lat1, lng1)$ 的点，且飞行高度为 h . 求能看到北极星的路程长度与总路程长度之比。

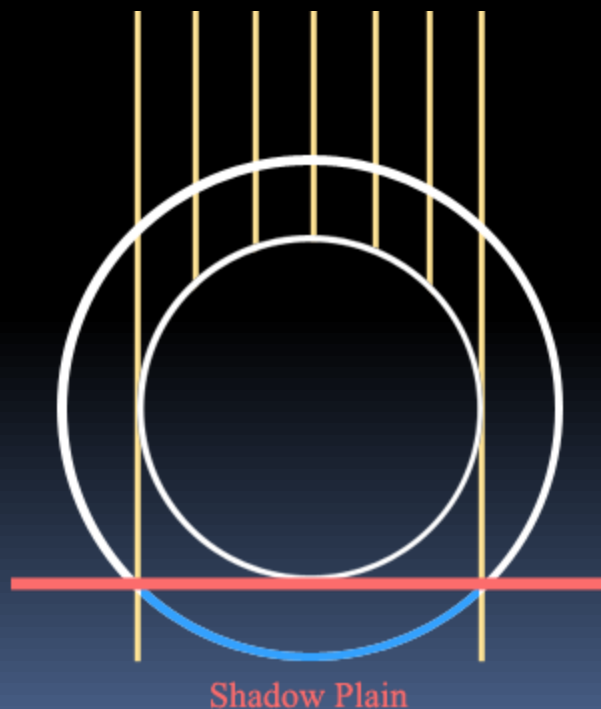
解题思路

- 飞行是在一个半径为 $r + h$ 的球上进行的，看不到北极星的部分如图所示。



解题思路

- 这货可以处理成在某个“影子平面”下面，北极星是不可见的。前提是只在大圆上飞行。



怎样求大圆上两点的距离

- 设球的半径是 R . 先求这两个点的坐标, 然后求圆心到它们的两个矢量的夹角 θ , 之后 $R\theta$ 即为所求答案。

怎样求大圆与平面的交点

- 然后需要处理路径从何时进入不可见区域。这需求大圆与平面的交点。
- 由于整个大圆在一个平面上，我们可以求大圆平面与原平面的交线，然后求交线与大圆所在球的交点。

收尾……

- 如果起讫点都看不到北极星，那就哪儿都看不到北极星。
- 否则，如果路径所在大圆与影子平面没有交点，那就全程都能看见北极星。
- 否则，起讫点与两个交点都在一个大圆上，根据它们之间的夹角讨论即可。
- 难度：较低

Bohemian Rhapsody

「波西米亚狂想曲」

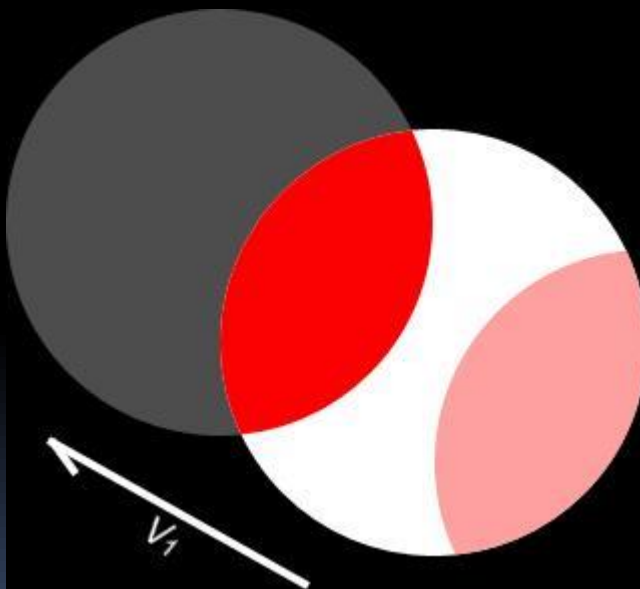
- POJ 2824
- 给出一个单位圆和若干（1000 个）矢量，每个矢量都对应一笔钱。
- 游戏一开始随机抽取单位圆中的一点，然后按顺序对每个矢量进行如下操作：首先将这个点按矢量平移，若这次平移之后该点仍在单位圆内，则获得这个矢量对应的那笔钱，否则游戏结束。每次平移之后不将点移回，然后试验下一个矢量。
- 求能够获得的金额的期望。

可行区域

- 基本想法：对于每一步，维护可行区域。
- 一开始的可行区域是整个圆。

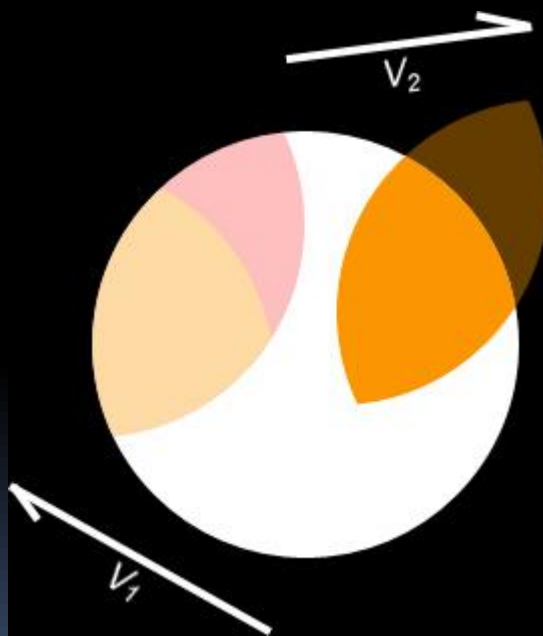
平移

- 考虑沿着矢量 v_1 平移之后的结果——



平移

- 然后再沿着矢量 v_2 平移——



平移

- 所以，每一步的可行区域，都是上一步的可行区域沿这一步的矢量平移之后，与原来圆的交。
- 考虑把移动可行区域变为移动圆本身。则每一步的可行区域，都是上一步的可行区域与某一个新的圆的交。

概率与期望

- 获得了每一步的可行区域之后，就可以求它的面积，然后计算在这一步之后还能拿钱的概率。
- 从而即可算出期望。

圆交的增量算法

- 每次新增加一个圆的话，可以仿照半平面交，给出圆交的增量算法。
 - ▣ 简单来说就是用新的圆和原来边界上的圆求交，然后求得新的交。每次 $O(n)$ 。
- 而且本题所有的圆的半径都是一样的。讨论起来比较方便。
- 十分考验基本功的题。



Telescope

「望远镜」

- POJ 3675
- 求简单多边形与圆的交的面积。

叉积与圆的有向面积交

- 凸多边形的面积，实际上是叉积的有向面积的和。所以只要能求出来每个叉积与圆的有向面积交即可。
- 而叉积就是一个三角形。
- 叉积的原点可以取圆心，这样三角形的一个顶点就在圆心了。

Case 1

- 三角形的两条边长全部短于半径。



Case 2

- 三角形的两条边长全部长于半径，并且另一条边与圆心的距离也长于半径。



Case 3.1

- 三角形的两条边长全部长于半径，但是另一条边与圆心的距离也短于半径。
- 垂足落在第三条边上。



Case 3.2

- 三角形的两条边长全部长于半径，但是另一条边与圆心的距离也短于半径。
- 垂足落在第三条边外。



Case 4

- 三角形的两条边长一条长于半径，一条短于半径。





有向面积

- 既然是有向面积，求交的面积的时候要考虑叉积的符号。

Shadow 「影子」

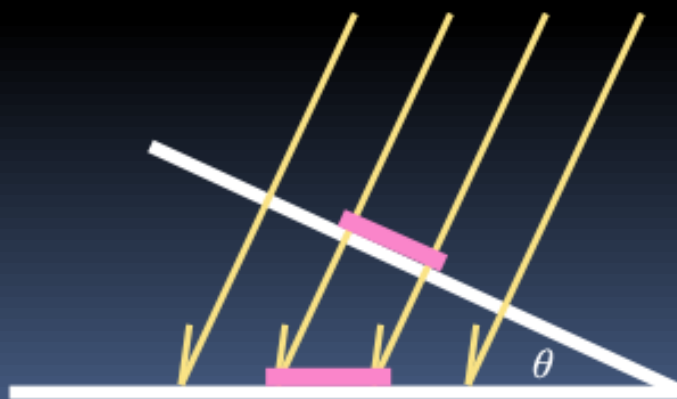
- POJ 4010 / HDU 4130
- 天上有若干个球和一个凸多面体。求在给
定平行光下这些东西的投影的面积。
- 球的数目 N 以及凸多面体的顶点数 M 之和
不超过 500。

地上是什么东西

- 凸多面体的投影是凸多边形。
- 所以将凸多面体的顶点都投到地上，然后求凸包就是凸多面体的投影。
- 可是球的投影是椭圆……
- 难道说就到此为止了么 $>$ $<$

换个投影面

- 我们换一个与光线垂直的平面进行投影，这样球的投影就是圆了。
- 但是原来的面积怎么办？
- 事实上，新投影面上的面积与旧投影面上的面积之比为 $\frac{1}{\cos \theta}$ 。



噩梦的开始

- 于是就变成了一个凸多边形和一堆圆的面积并。
- 按照原先所讲的算法做即可。
- 但是那个编码复杂度……你们懂的。

- 传送门：

<http://hi.baidu.com/bilidu/item/223b4294ab1b2e4bf042153c>

- 难度：高

LL's Cake

「LL 的蛋糕」

- POJ 3743
- 一个半径为 10 的圆蛋糕，切了 N ($N \leq 100$) 刀，而且保证没有三刀交于一点。
 - ▣ 每一刀都是给定圆上两点确定的。
- 求里面最大一块的面积是多少。

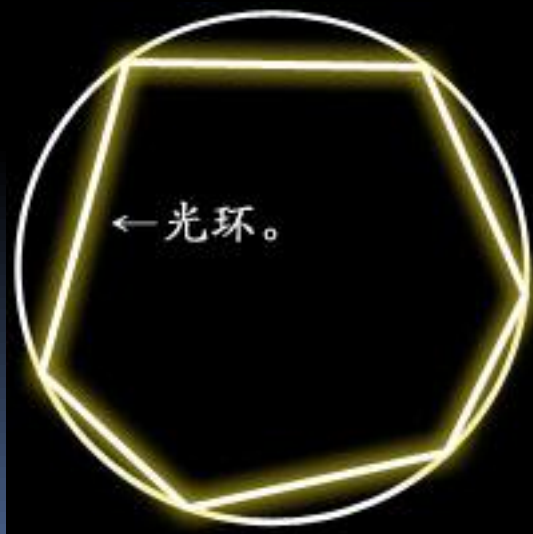


很像半平面交

- 不是么？
 - 但是外围是个圆，也就是说会有一些弓形。
- 

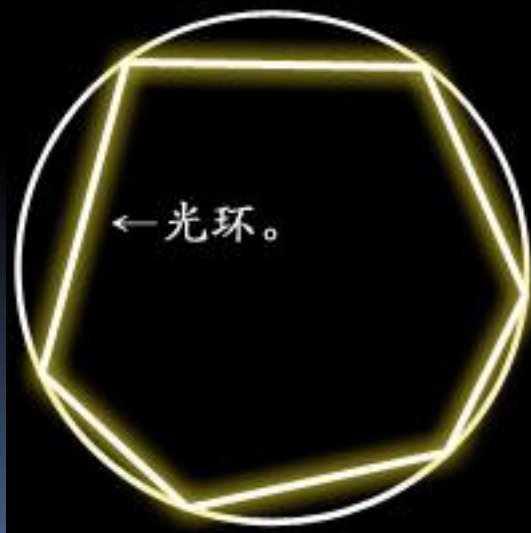
“光环”

- 首先离散化所有的角度，然后以这些角度为顶点建立初始多边形。
- 然后，给所有的边一个“光环”——



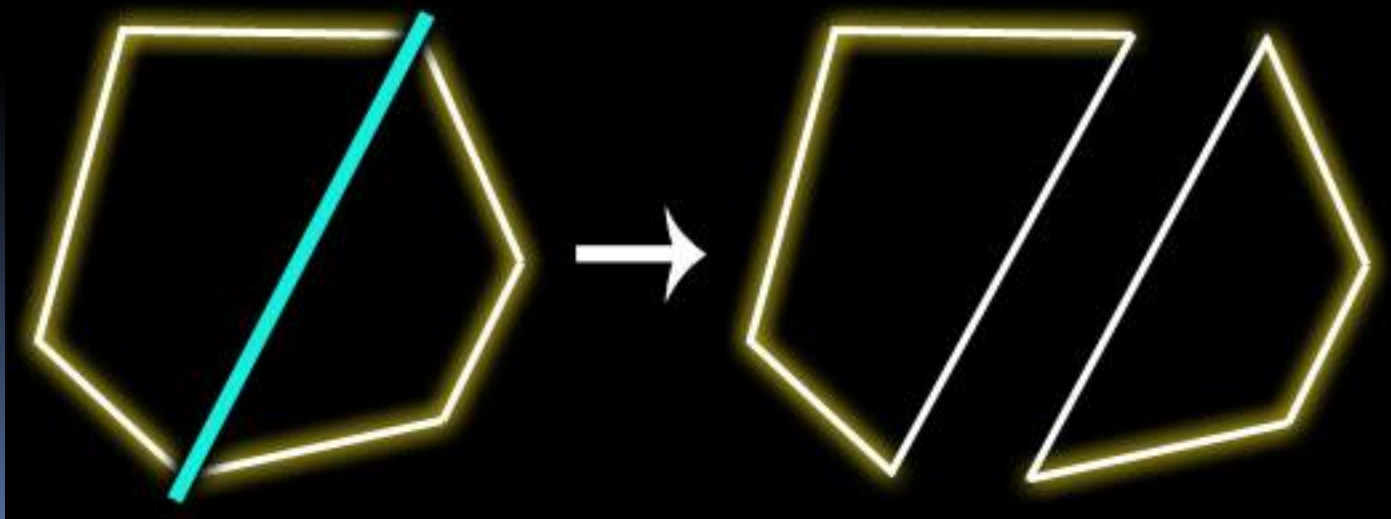
“光环”

- 所谓“光环”，其实就是线段的附加属性，表示这条线段对应的弓形的面积。
- 不难发现，这些线段都不会被切断。



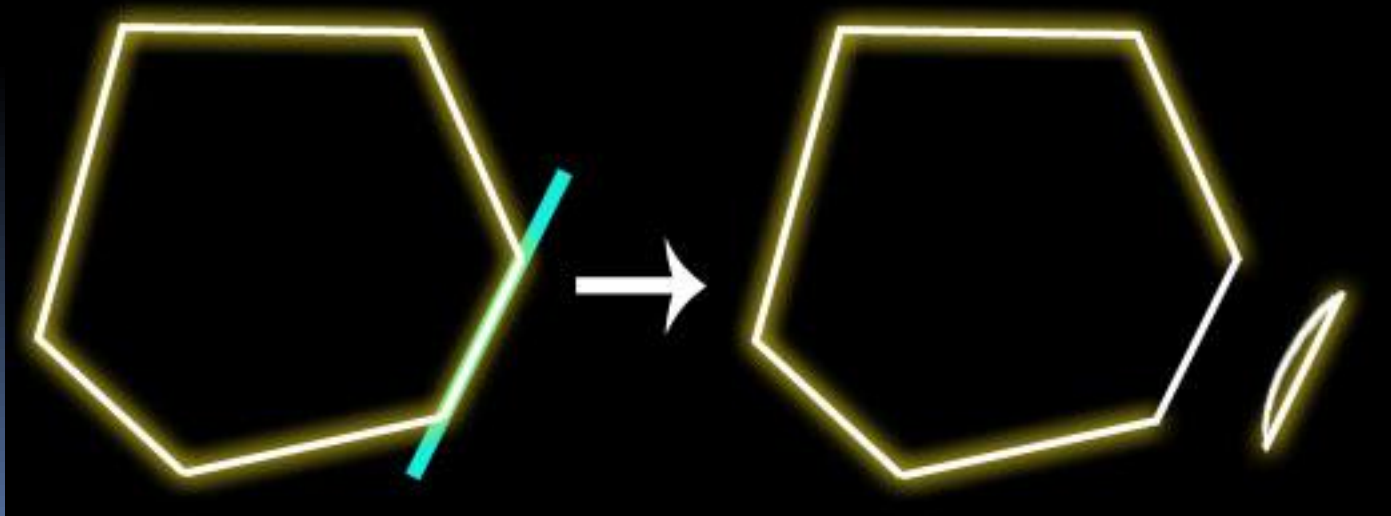
朴素半平面交

- 然后就转化成了带“光环”的半平面交。
- 切的时候，新产生的线段都是不带光环的，或者说可以说光环的大小是 0。



边界情况

- 不过这种半平面交新增了一种边界情况。如果某一刀与多边形的一条边重合，要切出一个只有光环的面积是 0 的多边形，对应原来的弓形。



加速指南

- 由于一个圆切 N 刀最多会产生的块数是 $O(N^2)$ 级别的，每块边数是 $O(N)$ 级别的，从而每次切的复杂度是 $O(N^3)$ 级别的，总复杂度是 $O(N^4)$ 级别的。
- 一个简单但是效果出色的剪枝：由于只求最大的一块，而最大的一块的面积肯定不会小于最大的一个弓形，所以比最大的弓形面积小的多边形可以舍弃。

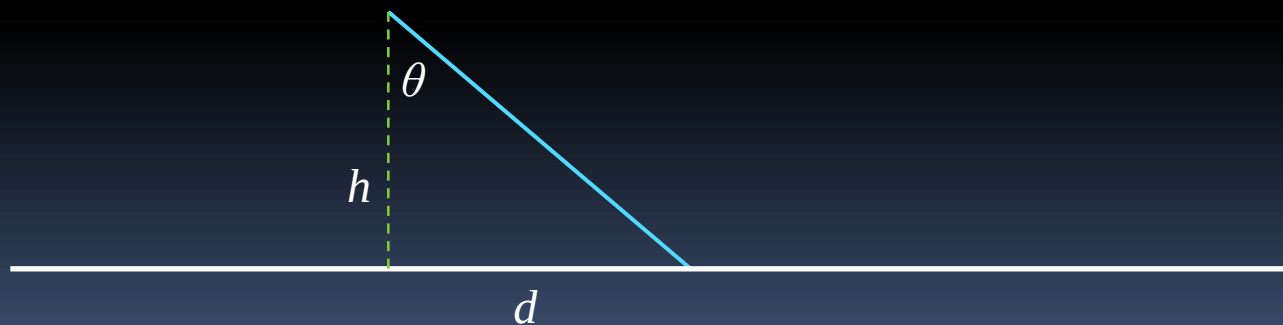
Highway

「自驾畅游美国大平原」

- HDU 5134
- 你刚刚领了年终奖。现在你有钱了，你打算去美帝自驾游 20 天。不过，你一不小心从高速公路上开下去了。当你发现的时候，你已经离高速公路 D 英里了。
- 你在高速上可以开 v_1 mph，而在外面只能开 v_0 mph（由于精度问题，保证 $v_1 > 1.2v_0$ ）。
- 高速公路是无限延伸的直线。求你现在在 T 时间内驾车能覆盖的区域的面积。

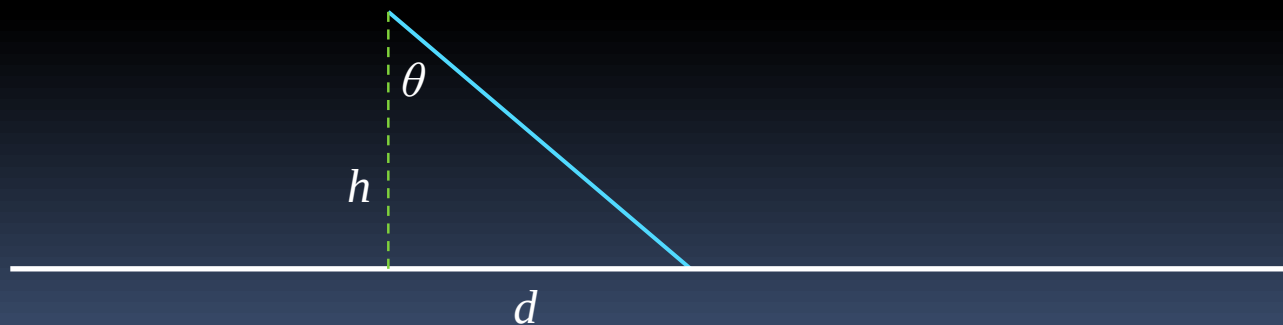
上高速

- 我们研究一下如何上高速才是最快的。假定你要上高速去高速上的某无限远点。



上高速

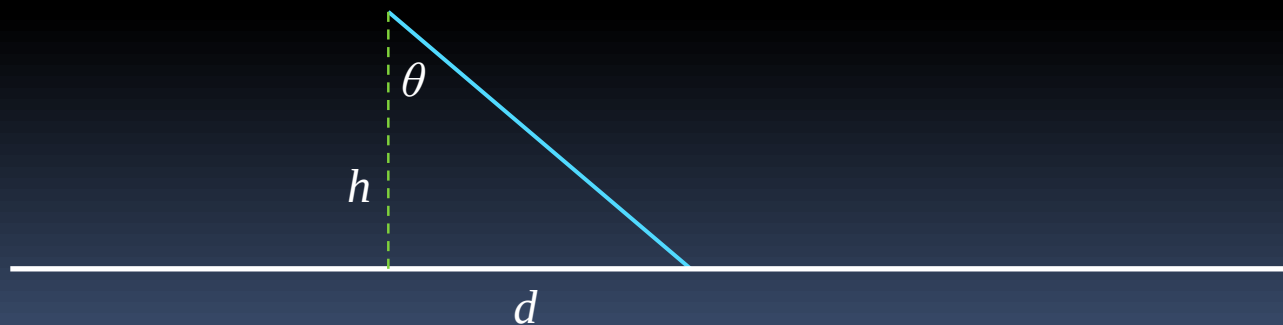
- $t = \frac{\sqrt{h^2 + d^2}}{v_1} + \frac{L-d}{v_0}$, L 是充分大的常数
- $\frac{dt}{dd} = \frac{d}{v_1 \sqrt{h^2 + d^2}} - \frac{1}{v_0}$



上高速

$$\frac{dt}{dd} \geq 0 \Leftrightarrow \frac{d}{\sqrt{h^2 + d^2}} \geq \frac{v_1}{v_0} \Leftrightarrow \sin \theta \geq \frac{v_1}{v_0}$$

- 从而 $\sin \theta = \frac{v_1}{v_0}$ 的时候用时最短。



最短路

- 从而 AB 两点之间的最短路由下图所示——
 - 如果 B' 在 A' 左边，那么最短路是直接开过去。



覆盖面积

- 如果不走高速公路，那显然是个圆形。
- 如果走高速，由前面的分析，边界应该是直线。
- 由于它沿 x 轴和 y 轴对称，所以应该是菱形。
- 然后就变成了菱形和圆形的面积并。

Box

「最终鬼畜杀菌计划」

- [BZOJ 2602](#)
- 栋栋有一个凸多面体形状的漂亮盒子，这个盒子的每个面都由形状为凸多边形的透明玻璃薄壁组成。
- 每隔一段时间，栋栋就要使用一个激光发射器给他的盒子杀菌。激光发射器能发射出一束激光，当激光照射到盒子的玻璃薄壁时，会把薄壁内外两面的细菌杀死，同时有大量光线透射过玻璃，少量光线被玻璃反射。透射过玻璃的光线沿原方向前进，反射的光线遵循光线的镜面反射定理，即在反射时反射光线、入射光线和镜面的法线在同一平面内，反射光线与入射光线分居法线两侧，反射角（反射光线与法线的夹角）等于入射角（入射光线与法线的夹角）。

Box

「最终鬼畜杀菌计划」

- 我们认为激光被玻璃反射 k 次后的能量不足以杀死细菌（即使有多个这样的光线同时照射在玻璃薄壁上也不能杀死细菌），而 k 次反射前激光照射到的细菌都会被杀死。
- 激光发射器的发射口是三角形的，当发射激光时，整个发射口都会发出激光。
- 给定盒子和激光发射器的位置，栋栋想知道，盒子的玻璃薄壁上有多大面积的细菌被杀死了。由于盒子的内外两面的细菌必然是同时被杀死或者同时不被杀死，所以只要算出外面被杀死的细菌面积即可。
- 注意，激光或经过反射的激光可能与某些表面平行，但本题的数据保证激光或经过反射的激光不会照射到与之平行的表面。

光线队列

- 光线就是一个凸多边形，加一个方向矢量。
- 用一个队列处理光线。
- 由于光线有透射，要枚举每一个面。先判断这个面与光线是否有交（看方向），再将光线的凸多边形投影到面上，之后求光线与盒子的这个面的交，求光线反射后的方向，最后将处理完毕的光线加入队列。
- 反射 k 次的光线不继续反射。

求被照射过的面积

- 对于每个面和每条合法光线，求所有光线与这个面交出的凸多边形，然后求这些凸多边形的并集与原来的面的交。
- 这个面积可以用 Simpson 来求。

启示录

- 作为 CTSC 2011 的压轴题，本题的难度是可想而知的。
- 这就要求选手在处理此类问题的时候**不慌不乱**，**小心谨慎**，**不怕编写和调试漫长的代码**，**还要有足够的手速和超强的耐心**。
- 难度：极高